



计算机系统实验报告

实验三：应用开发

姓 名： 王亦玮

学 号： 2351274

指导教师： 郭玉臣

实验日期： 2026 年 7 月 4 日

同济大学 计算机科学与技术学院

目录

1	引言	3
2	时钟生成与复位管理	3
2.1	时钟频率的确定与实现	3
2.2	复位信号的合成	4
2.3	Wishbone 总线互联与地址映射	4
3	SPI Flash 控制器	5
3.1	硬件接口与时钟输出	5
3.2	状态机设计	5
3.3	SPI 数据采样的相位修正	6
4	DDR2 SDRAM 控制器	7
4.1	MIG IP 核的配置	7
4.2	协议转换桥接	7
4.3	BootLoader 中 DDR2 就绪轮询的旁路	7
5	串口通信与通用输入输出	8
5.1	UART16550 串口控制器	8
5.2	GPIO 控制器	8
5.3	中断汇聚	9
6	操作系统移植与编译部署	9
6.1	启动流程	9
6.2	μ C/OS-II 的体系结构适配	10
6.3	交叉编译与映像生成	10
6.4	应用层开发：2048 与走迷宫	11
6.4.1	协同架构与数据协议	11
6.4.2	2048 游戏	11

6.4.3	走迷宫游戏	12
6.4.4	PC 端图形界面	12
7	下板验证	13
7.1	部署与启动	13
7.2	基础串口输出	13
7.3	数码管与 LED 运行状态	14
7.4	应用层验证：2048 与走迷宫	16
8	总结	18
A	关键模块源代码	20
A.1	SoC 顶层模块	20
A.2	SPI Flash 控制器	29
A.3	Wishbone 总线接口	38
A.4	全局宏定义	42
A.5	DDR2 控制器封装	49
A.6	引脚约束	56

1 引言

本实验在实验一所完成的 OpenMIPS 五级流水线处理器基础之上，构建一套完整的片上系统（SoC），并在其上成功部署 $\mu\text{C}/\text{OS-II}$ 实时操作系统。实验一所交付的 CPU 以 FPGA 片内 Block RAM 作为指令与数据存储器，所有访存操作在单周期内完成，且不具备与外设交互的能力。这种封闭式架构无法满足嵌入式操作系统的运行需求——操作系统内核镜像需要非易失存储介质固化，运行时数据需要大容量动态内存承载，调试信息则需要通过串行接口输出至外部终端。

为此，本实验引入 Wishbone B4 片上总线作为互联标准，依次挂载 SPI Flash 控制器（程序固化存储）、DDR2 SDRAM 控制器（运行时内存）、UART16550 串口控制器（调试日志输出）以及 GPIO 控制器（驱动板载数码管与 LED）。硬件平台搭建完毕后，基于 MIPS M14K 的已有移植代码对 $\mu\text{C}/\text{OS-II}$ 进行体系结构适配，利用 mips-sde-elf 交叉编译工具链生成二进制映像，烧录至 SPI Flash 并在 Nexys4 DDR 开发板上完成功能验证。在操作系统稳定运行的基础上，进一步开发了 2048 与随机迷宫两款交互式应用，并配套实现 PC 端图形界面，构成“板载引擎 + 图形前端”的协同架构。

硬件开发环境为 Windows 11 下的 Vivado 2024.2，软件编译环境为 Ubuntu 12.04 虚拟机中的 mips-sde-elf (GCC 4.3) 工具链。目标板卡搭载 Xilinx Artix-7 xc7a100tcsg324-1 FPGA，板载 DDR2 颗粒型号为 MT47H64M16HR-25:H（128 MB），SPI Flash 型号为 Spansion S25FL128S（16 MB）。

2 时钟生成与复位管理

2.1 时钟频率的确定与实现

Nexys4 DDR 板载 100 MHz 有源晶振连接至 FPGA 全局时钟引脚 E3。设计初期曾尝试将 100 MHz 直接用作全系统单一工作时钟，但 Vivado 布局布线后的时序分析报告显示，DDR2 MIG IP 核周边的组合逻辑路径无法在 10 ns 时钟周期内完成信号传播，最差负裕量（WNS）达 -5.150 ns。

解决方案为通过 Vivado Clocking Wizard IP 核（底层 MMCME2_ADV 原语）对输入时钟进行 PLL 分频。MMCM 的关键配置参数见表 1。

表 1: MMCM 配置与时钟分配

参数/信号	设定值	说明
DIVCLK_DIVIDE	1	输入分频, $VCO = 100 \times 10/1 = 1000\text{ MHz}$
CLKOUT0_DIVIDE	20	输出 50 MHz, 供 CPU、总线及所有外设
CLKOUT1_DIVIDE	5	输出 200 MHz, 供 DDR2 MIG 参考时钟
时钟域	频率	供给对象
clk_out1	50 MHz	CPU 流水线、Wishbone 总线、UART、GPIO、Flash
clk_out2	200 MHz	DDR2 MIG IP clk_ref_i

系统主时钟降至 50 MHz 后，此前全部违规路径均通过建立/保持时间检查。

2.2 复位信号的合成

系统复位初始方案仅依赖板载按钮 `rst_n` (C12 引脚，低有效)。上电测试中观察到系统行为存在间歇性不一致。排查后确认，FPGA 配置完毕后 MMCM 锁相过程（通常持续数百微秒至数毫秒）可能尚未完成——若复位在输出时钟稳定前释放，CPU 将在不稳定时钟沿上执行不确定的操作。

修复方案为将 MMCM 的 PLL 锁定指示 `locked` 纳入复位逻辑：

```
1 assign rst = ~rst_n | ~locked;
```

Listing 1: 复位信号合成

仅当板载按钮释放 (`rst_n = 1`) 且 MMCM 完成锁相 (`locked = 1`) 时，`rst` 方翻转为 0。此修改消除了所有上电初期的不可复现异常。

2.3 Wishbone 总线互联与地址映射

CPU 与各外设之间通过 Wishbone B4 共享总线互联。总线矩阵采用 OpenCores 的 WB_CONMAX 模块，以地址信号高 4 位作为从设备选择依据。各从设备的地址区间见表 2。

表 2: 从设备地址映射

地址范围	addr[31:28]	从设备	功能
0x0000_0000 -- 0x0FFF_FFFF	4'b0000	DDR2 SDRAM	256 MB 运行时内存
0x1000_0000 -- 0x1FFF_FFFF	4'b0001	UART16550	串口通信寄存器组
0x2000_0000 -- 0x2FFF_FFFF	4'b0010	GPIO	I/O 寄存器组
0x3000_0000 -- 0x3FFF_FFFF	4'b0011	SPI Flash	16 MB 程序存储

CPU 复位向量固定指向 0x3000_0000，即 Flash 起始地址。CPU 与 Wishbone 总线之间的协议转换由 `wishbone_bus_if` 模块完成，在处理器顶层中例化两次——分别服务取指通路和数据访存通路。模块内部以三态状态机运作：WB_IDLE 状态下收到 CPU 侧 `cpu_ce_i` 有效后发起 Wishbone 周期（置 `stb_o` 和 `cyc_o`）并向 `ctrl` 模块请求流水线暂停；WB_BUSY 状态下等待从设备 `ack_i` 应答，收到后将读数据锁存至 `rd_buf` 并释放总线周期；若此时流水线仍处于 `stall`，则进入 WB_WAIT_FOR_STALL 状态直至暂停释放。`ctrl` 模块统一汇集取指 WB、ID 数据相关、EX 多周期指令及 MEM 访存 WB 四个暂停源的 `stallreq` 信号，生成 6 位 `stall` 向量分发至各级流水线寄存器。

3 SPI Flash 控制器

3.1 硬件接口与时钟输出

本系统以 Nexys4 DDR 板载的 Spansion S25FL128S（128 Mbit / 16 MB SPI NOR Flash）作为程序固化存储器件。该器件通过四线 SPI 接口与 FPGA 通信——`sck`（串行时钟，FPGA 输出）、`cs_n`（片选，低有效，接 L13）、`sdo`（MOSI，命令/地址/写数据，接 K17）、`sdi`（MISO，读数据，接 K18）。

S25FL128S 的 SCK 引脚在板卡上接至 FPGA 专用配置时钟引脚 CCLK，普通 I/O 在用户模式下不可直接驱动。解决途径为调用 Xilinx Artix-7 内建的 STARTUPE2 原语——将其 USRCCLKO 端口接至内部逻辑生成的 `sck` 信号，USRCCLKTS 接地以持续使能输出通道，USRDONETS 接高将 DONE 引脚置为高阻态。

3.2 状态机设计

Flash 控制器的核心行为由一台包含 13 个状态的有限状态机定义，覆盖从接收 Wishbone 总线请求到返回完整 32 位读数的全过程。各状态的功能见表 3。

表 3: Flash 控制器状态转移表

状态	阶段	操作
IDLE	等待	等待 <code>cyc_i & stb_i</code> 有效后无条件跳转至 START
START	启动	锁存地址，使能 SCK，拉低 <code>cs_n</code>
INST_OUT	命令	串行移出 READ 操作码 0x03 (8 位)
ADDR1_OUT	地址	移出 <code>addr[23:16]</code>
ADDR2_OUT	地址	移出 <code>addr[15:8]</code>
ADDR3_OUT	地址	移出 <code>addr[7:0]</code>
READ_DATA1	数据	接收第 0 字节流，奇数拍移位采样
READ_DATA2	数据	接收第 1 字节， <code>sdo_count==2</code> 时锁存至 <code>read_data[31:24]</code>
READ_DATA3	数据	接收第 2 字节，锁存至 <code>read_data[23:16]</code>
READ_DATA4	数据	接收第 3 字节，锁存至 <code>read_data[15:8]</code>
READ_DATA5	数据	接收第 4 字节 (冗余)，锁存至 <code>read_data[7:0]</code>
WAITING	结束	关闭 SCK 使能，拉高 <code>cs_n</code>
ENDING	应答	通过 Wishbone 返回 <code>read_data</code> 并置 <code>ack_o</code>

IDLE 至 START 设计为无条件跳转——原始设计中该处的 `flash_continue` 门控信号因无法在 Flash 作为唯一取指源的架构中获得有效驱动，移除后方便 CPU 正常启动。

3.3 SPI 数据采样的相位修正

系统集成调测阶段发现，CPU 虽能成功发起取指总线事务（调试 LED 之 LD1 亮起可证），但读出数据始终错误，串口无任何输出。在 Vivado 中插入 ILA IP 核对 `state`、`sdo_count`、`sck`、`sdi`、`datain_shift` 及 `read_data` 等关键节点进行波形捕获。

ILA 首帧波形即揭示故障：期望的首条指令机器码为 0x3C011000 (`lui $1, 0x1000`)，`read_data` 实际最高字节为 0x78，恰为正确值 0x3C 左移一位。追踪 `datain_shift` 逐拍演变确认，`state`（经两级寄存器到达判断逻辑）与 `sdo_count`（仅经一级）之间存在一个时钟周期的偏差——Flash 在 `sdo_count == 0` 时即输出最高数据位，但 `state` 仍停留在 ADDR3_OUT，采样逻辑未激活，最高位被漏采，此后所有位整体左偏一位。

修正措施为将采样边沿从偶数拍改为奇数拍（`sdo_count[0] == 1` 时移位），字节锁存时刻从 `sdo_count == 0` 改为 `sdo_count == 2`。各 READ_DATA 状态中，四字节的锁存位置分别为：READ_DATA2 锁存 [31:24]、READ_DATA3 锁存 [23:16]、READ_DATA4 锁存 [15:8]、READ_DATA5 锁存 [7:0]。修正后 `read_data` 正确捕获首条指令，CPU 恢复正常执行，串口随即输出启动日志。ILA 修正后波形见图 1。

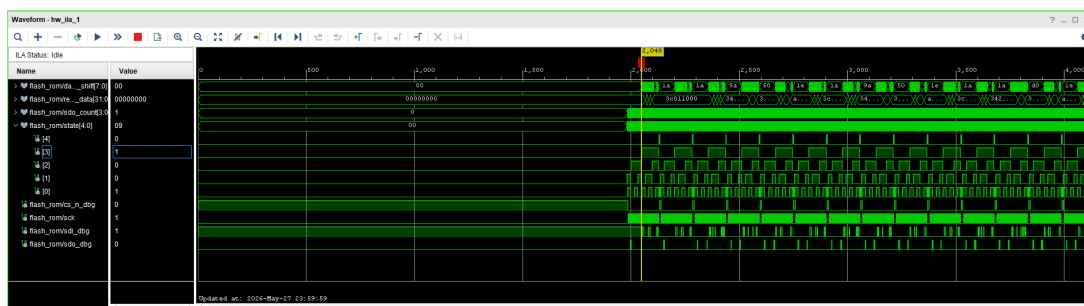


图 1: 修正采样相位后 datain_shift 从 0x00 逐步累积至 0x3C

4 DDR2 SDRAM 控制器

4.1 MIG IP 核的配置

DDR2 SDRAM 的物理层操作——上电初始化序列、ZQ 阻抗校准、读写眼图训练、周期性刷新——由 Xilinx Memory Interface Generator (MIG) IP 核承担。MIG 在本实验中配置为 DDR2 模式，外部数据总线 16 位，突发长度 8（对应每次 128 位数据传输），APP 用户接口数据宽度 128 位、地址宽度 27 位，PHY 至控制器时钟比例为 4:1。所需 200 MHz 参考时钟由 Clocking Wizard 的 `clk_out2` 提供，核内 PLL 在此基础上生成 DDR2 PHY 的 400 MHz 双沿时钟及用户侧 `ui_clk`。

FPGA 配置完成后 MIG 自动执行校准序列，校准完成时 `init_calib_complete` 信号翻转为高，标志 DDR2 进入可正常读写状态。

4.2 协议转换桥接

MIG 的 APP 用户接口与 Wishbone 总线之间存在协议差异，本设计以两级桥接解决这一不匹配。底层为 Digilent 公司 Looper 项目中的 Ram2Ddr 模块（VHDL 实现），将 MIG APP 接口封装为一组类 SRAM 简易接口（`ram_a`、`ram_dq_i/o`、`ram_cen/oen/wen`）。顶层为自编写的 DDR2 包装模块（DDR2.v，Verilog 实现），完成 Wishbone 端 32 位单次读写到 Ram2Ddr 接口的最终转换，包括：写操作时对齐拼入 128 位写数据总线，读操作时从 128 位返回数据中截取对应 32 位字段，以及在 MIG 忙碌时正确处理 `app_rdy` 和 `app_wdf_rdy` 反压信号。

4.3 BootLoader 中 DDR2 就绪轮询的旁路

BootLoader 启动代码的偏移量约 0x60 处存在一段轮询：反复读取 GPIO 输入寄存器第 16 位，该位为 1 后方可继续将 OS 映像搬移至 DDR2。其物理含义是等待 MIG 的

`init_calib_complete` 信号置位。若 GPIO 至该信号的连接出现故障，轮询将陷入死循环，系统表现为上电后完全卡死。

考虑到 MIG 校准在 FPGA 配置后自动执行，且实际耗时远小于 CPU 从 Flash 读取并执行至轮询点的时间，在 SoC 顶层将该位强制置为高电平不会引入时序竞态：

```
1 assign gpio_i_temp = {15'h0, 1'b1, gpio_i};
```

Listing 2: 顶层 GPIO 输入第 16 位固定上拉

此修改使 BootLoader 首次读取该位即满足条件进入搬移流程，消除了因信号连接异常导致启动死锁的隐患。

5 串口通信与通用输入输出

5.1 UART16550 串口控制器

UART 控制器采用 OpenCores 社区的 UART16550 IP 核，挂接于 Wishbone 从设备端口 1。该 IP 核实现了完整的 16550 兼容寄存器组，主要寄存器见表 4。

表 4: UART16550 主要内部寄存器

寄存器	偏移 (DLAB=0)	功能
RBR (接收缓冲)	0x0 (只读)	存放接收到的数据字节
THR (发送保持)	0x0 (只写)	写入待发送的数据字节
IER (中断使能)	0x1	控制四类中断的使能位
LCR (线路控制)	0x3	字长、停止位、校验位配置
LSR (线路状态)	0x5 (只读)	数据就绪、发送寄存器空等状态位
DLL / DLM	0x0 / 0x1 (DLAB=1)	波特率分频锁存器

对于 50 MHz 系统时钟与 4800 bps 目标波特率，分频系数 $50 \times 10^6 / (16 \times 4800) \approx 651$ 。数据帧格式为 8-N-1 (8 位数据、无校验、1 位停止)。串口物理引脚中，TXD 连接至板卡 D4 引脚 (JD1)，RXD 连接至 C4 引脚 (JD2)，经 FT2232H USB-UART 桥接芯片后在 PC 端枚举为 COM3 虚拟串口。调试时使用 SuperCom 软件以 4800-8N1 参数连接。

5.2 GPIO 控制器

GPIO 控制器采用 OpenCores 的 GPIO IP 核，配置为 32 位双向模式，挂接于 Wishbone 从设备端口 2。内部含 7 个寄存器 (输入、输出、输出使能、中断使能、中断

触发类型、辅助控制、中断状态)，通过 Wishbone 地址低若干位索引。GPIO 输出总线的低 8 位经约束文件映射至七段数码管的 8 根阴极段码引脚（CA–CG 及 DP），CPU 写入 GPIO 输出寄存器的低字节直接决定各段的亮灭。GPIO 输入的低 16 位连接至板载 16 个滑动开关，第 16 位在顶层被固定为高电平。

系统顶层额外增设了 9 根调试 LED 信号，由顶层逻辑直接驱动而不经 GPIO 模块，其定义见表 5。

表 5: 调试 LED 信号定义

编号	驱动信号	亮起时的含义
LD0	26 位计数器 bit25	时钟存活（~0.75 Hz 心跳）
LD1	Flash 端 cyc & stb 检测	CPU 曾成功向 Flash 发起取指
LD2	UART 端 cyc & stb 检测	CPU 曾访问 UART 模块
LD3	sdram_init_done 直连	DDR2 MIG 校准完成
LD4–LD7	计数器 bit24–bit21	跑马灯——CPU 在持续运行
LD8	uart_out 直连	实时镜像 UART TXD 引脚电平

5.3 中断汇聚

三个外设中断源——GPIO 中断、UART 中断及 CP0 内部定时器中断——按 {3'b000, gpio_int, uart_int, timer_int} 格式拼接为 6 位向量接入 CPU 的 int_i 端口。CP0 协处理器依据 Status 寄存器的中断屏蔽位和 Cause 寄存器的中断待处理位决定异常触发时机。

6 操作系统移植与编译部署

6.1 启动流程

系统上电后的启动过程由固化在 Flash 起始地址 0x3000_0000 处的 BootLoader 主导，分为三个阶段，见表 6。

表 6: 系统启动三阶段

阶段	主要操作	执行位置
1. 就绪确认	读 GPIO 第 16 位, 等待 DDR2 校准完成	Flash 中
2. 映像搬移	将 $\mu\text{C}/\text{OS-II}$ 映像从 Flash 逐段复制至 DDR2	Flash 中
3. 控制权移交	执行跳转指令, PC 重定向至 DDR2 中的 OS 入口	Flash $\rightarrow$ DDR2

BootLoader 适配中涉及两处修改: UART 分频系数确认无需改动 (系统时钟 50 MHz + 波特率 4800 的组合所得系数 651 与 100 MHz + 9600 一致), 以及源码中通用标号 (L1、L2 等) 改为带唯一前缀的名称以避免链接时符号重定义。

6.2 $\mu\text{C}/\text{OS-II}$ 的体系结构适配

$\mu\text{C}/\text{OS-II}$ 中与处理器直接耦合的代码集中在三个文件中。`os_cpu.h` 定义与 MIPS 字长相关的数据类型、栈增长方向 (向下) 及临界区保护宏。`os_cpu_a.S` 用 MIPS 汇编实现 `OSCtxSw` (任务级上下文切换) 和 `OSIntCtxSw` (中断级上下文切换), 二者须完整保存/恢复 32 个通用寄存器、HI/LO 乘除寄存器及 CP0 的 Status/Cause/EPC 等关键状态。`os_cpu_c.c` 中的 `OSTaskStkInit` 为新任务构造初始栈帧, 使其首次被调度时能通过模拟的异常返回达到入口函数。

基础用户态测试程序 (`openmips.c`) 维护一个由定时器中断驱动的计数器 (每 10 次中断自增一次, 低 4 位写 GPIO 输出以更新数码管), 并通过 UART 逐字符循环发送预设字符串, 用以验证操作系统任务调度与外设访问的正确性。在此之上进一步开发了两款交互式应用, 详见 6.4 节。

6.3 交叉编译与映像生成

软件编译在 Ubuntu 环境中借助 `mips-sde-elf` (GCC 4.3) 工具链完成。编译流程见表 7。

表 7: 交叉编译与映像生成流程

步骤	工具/命令	产出
子目录编译	<code>make (common/、ucos/、port/)</code>	各模块 .o 目标文件
链接	<code>mips-sde-elf-gcc -Tram.ld -o ucosii.om ...</code>	ELF 可执行映像
二进制提取	<code>mips-sde-elf-objcopy -O binary</code>	<code>ucosii.bin</code> 纯二进制
头部合并	<code>BinMerge.exe -f ucosii.bin -o OS.bin</code>	完整固件 <code>OS.bin</code>

链接脚本 `ram.ld` 将全部段定位至 DDR2 地址空间，`-e 256` 设定入口偏移 256 字节。最终生成的 `OS.bin` 烧录至 SPI Flash 起始地址。值得指出的是，由于该工具链为 32 位 (i686) 程序，在 64 位 WSL/Ubuntu 中运行需额外安装 `libc6-i386` 兼容库；且在 WSL2 下访问 `/mnt` 挂载的 Windows 文件时，因 `drvfs` 返回的 `inode` 较大，会导致 32 位编译器 `stat` 失败（报 `Value too large for defined data type`），需将源码拷贝至 WSL 原生文件系统（如 `~`）下编译。

6.4 应用层开发：2048 与走迷宫

在操作系统稳定运行的基础上，进一步在 $\mu\text{C}/\text{OS-II}$ 之上开发了两款交互式应用——2048 数字合并游戏与随机迷宫游戏，并配套实现 PC 端图形界面，构成“板载游戏引擎 + PC 图形前端”的协同架构。

6.4.1 协同架构与数据协议

OpenMIPS 作为游戏引擎，负责全部游戏逻辑（状态维护、移动与合并判定、迷宫生成），并通过 UART 向 PC 推送结构化状态；PC 端 Python 程序负责接收、解析与图形渲染。两端约定以文本行为单位交换数据，字段以分隔符切分，既便于程序解析，也可用 SSCOM 直接人工核对。两类应用的协议见表 8。

表 8: 应用层结构化数据协议

应用	数据行格式
2048	B:v0,v1,...,v15 S: 分数 G: 胜负状态
迷宫	M:w0,w1,...,w63 P: 玩家坐标 E: 终点坐标 W: 是否到达

其中迷宫墙值采用 4 位编码（上 =1、右 =2、下 =4、左 =8），每个格子以一个 0–15 的整数完整描述其四面墙的存在情况。控制输入沿用 GPIO 滑动开关：`gpio_i[1..4]` 分别对应右、左、下、上四个方向，`gpio_i[0]`（按钮 N17）作为确认键。

6.4.2 2048 游戏

`openmips.c` 中维护 4×4 棋盘与累计分数，实现上、下、左、右四个方向的滑动与同值合并逻辑，合并时将新值累加入分数。每次有效移动后，由 `send_state()` 将棋盘 16 个格子的数值、当前分数与胜负标志以协议格式经 UART 输出：

```
1 void send_state(void) {
2     int i, j;
```

```
3     uart_print_str("B:");
4     for (i = 0; i < BOARD_SIZE; i++)
5         for (j = 0; j < BOARD_SIZE; j++) {
6             if (i > 0 || j > 0) uart_putc(',');
7             uart_print_int(board[i][j]);
8         }
9     uart_print_str("|S:"); uart_print_int(score);
10    uart_print_str("|G:"); uart_print_int(is_game_over());
11    uart_putc('\n');
12 }
```

Listing 3: 结构化状态发送 (send_state)

6.4.3 走迷宫游戏

迷宫规模为 8×8 ，采用深度优先回溯 (DFS) 算法随机生成。算法从起点出发，随机选择一个未访问的相邻格子并打通二者之间的墙，递归深入；当四面均无未访问邻居时回溯，直至所有格子被访问。该过程等价于在完整网格图上构造一棵生成树，因此生成的迷宫在数学上保证全连通且任意两点间恰有一条路径——既不存在无法抵达的孤岛，也不存在成环的回路。玩家从左上角 (0,0) 出发，目标是到达右下角 (7,7) 的终点；每次移动前查询当前格在目标方向上是否有墙，有墙则拒绝移动并提示。复位时刻的 GPIO 开关状态被用作随机种子，因此复位前拨动开关即可生成不同的迷宫布局。

6.4.4 PC 端图形界面

PC 端使用 Python 标准库 `tkinter` 配合 `pyserial` 实现图形界面。后台线程持续读取串口并按行解析，主线程依据棋盘或迷宫数据在 `Canvas` 上渲染彩色方块与墙体。为增强表现力，2048 界面端复现了与板载一致的移动算法，通过对比相邻两帧反推移动方向，从而播放方块滑动与合并弹出动画；迷宫界面则播放玩家方块的平滑移动动画。整个过程中板载程序无需为动画做任何改动，体现了结构化协议带来的端到端解耦优势——同样的固件，既可由 SSCOM 以文本形式查看原始数据，也可由 PC 图形界面渲染为富表现力的交互体验。

7 下板验证

7.1 部署与启动

硬件设计经 Vivado 综合、布局布线并生成比特流后,依次执行以下操作:(1) Vivado Hardware Manager 将 OS.bin 烧录至 SPI Flash 起始地址;(2) PC 端 SSCOM 以 COM3 / 4800 / 8N1 参数打开串口;(3) 将比特流下载至 FPGA。复位释放后,系统按第 6 章所述启动序列自动进入运行状态。

7.2 基础串口输出

下载完成后数秒,SSCOM 窗口即显示出 $\mu\text{C}/\text{OS-II}$ 的启动日志,见图 2。

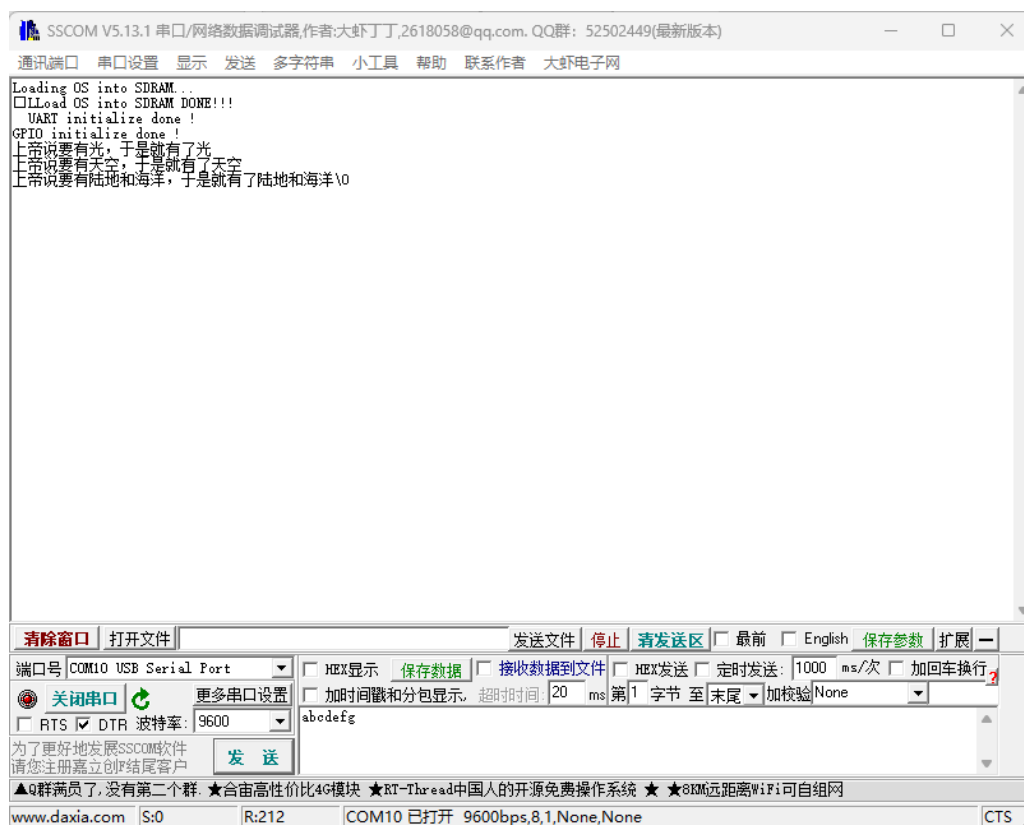


图 2: SSCOM 接收到的基础 OS 启动输出

输出中的关键信息——"Loading OS into SDRAM... DONE!!!"、"UART initialize done !" 和 "GPIO initialize done !"——分别印证了 BootLoader 映像搬运、UART 初始化和 GPIO 初始化三个环节的成功执行。

7.3 数码管与 LED 运行状态

$\mu\text{C}/\text{OS-II}$ 启动后，八位数码管同步显示递增数值（11111111 $\rightarrow$ 22222222 $\rightarrow \dots$ ），递增节奏均匀无卡顿。该行为由用户任务中定时中断驱动的计数器经 GPIO 输出寄存器刷新实现。实物状态见图 3 和图 4。

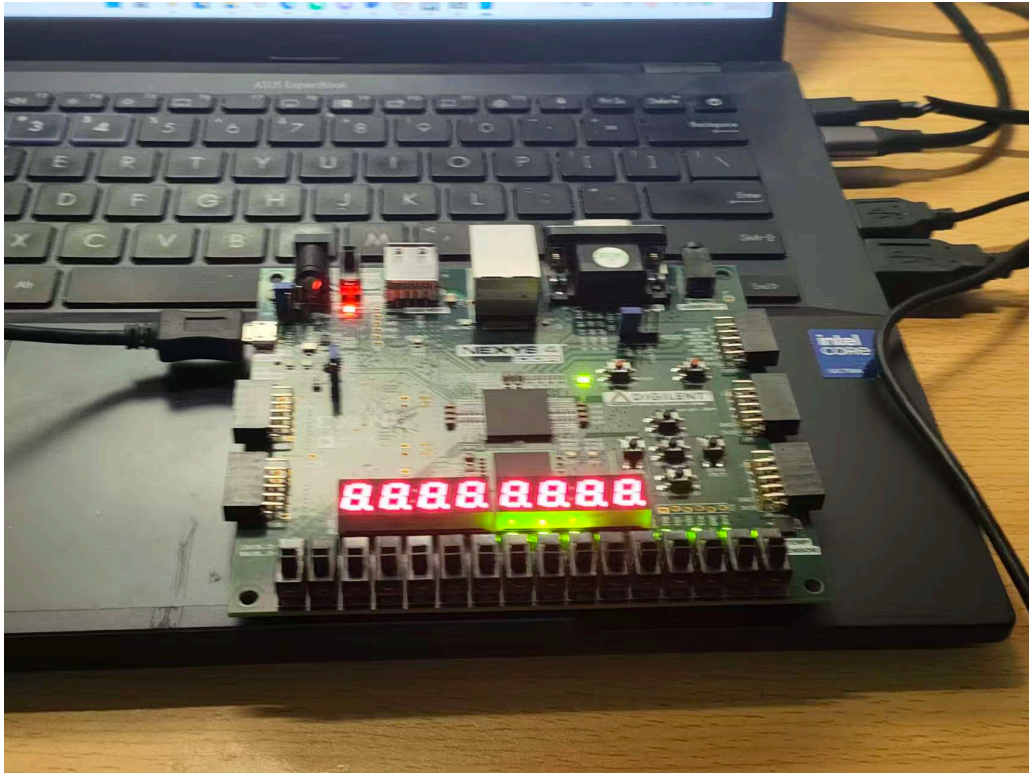


图 3: 数码管递增显示（状态一）

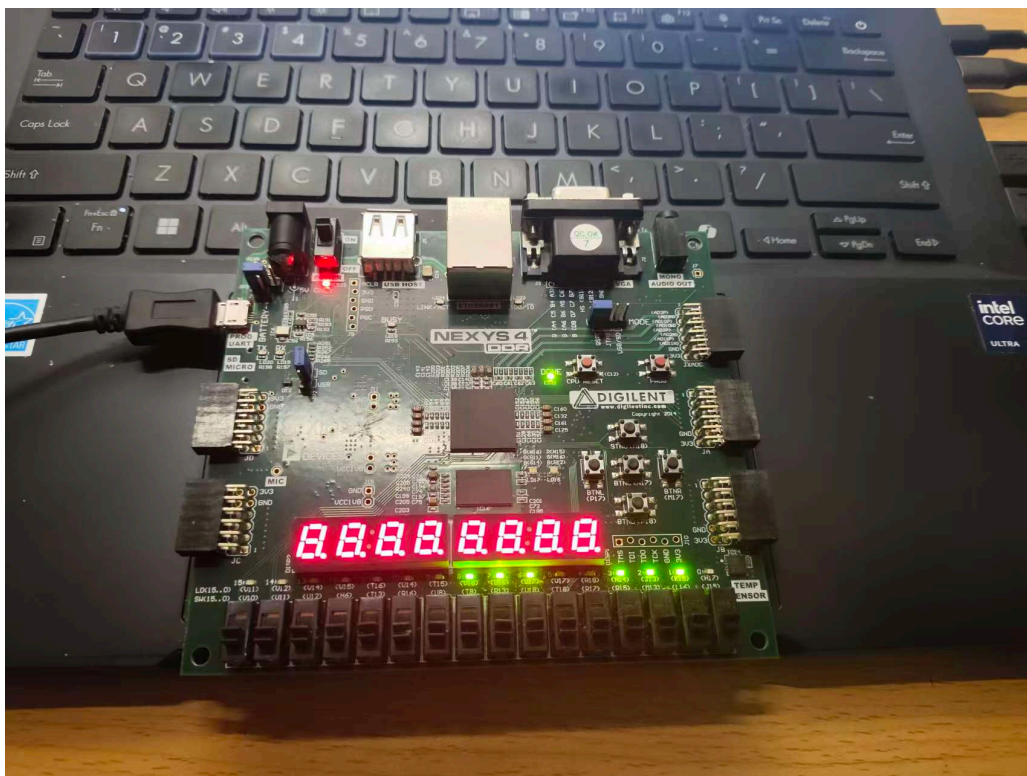


图 4: 数码管递增显示（状态二）

板载 9 颗调试 LED 的实测表现与表 5 所列设计预期一致（图 5）：LD0 以约 1.5 s 周期稳定闪烁，LD1/2/3 保持常亮，LD4–7 呈跑马灯效果，LD8 随串口输出闪烁。

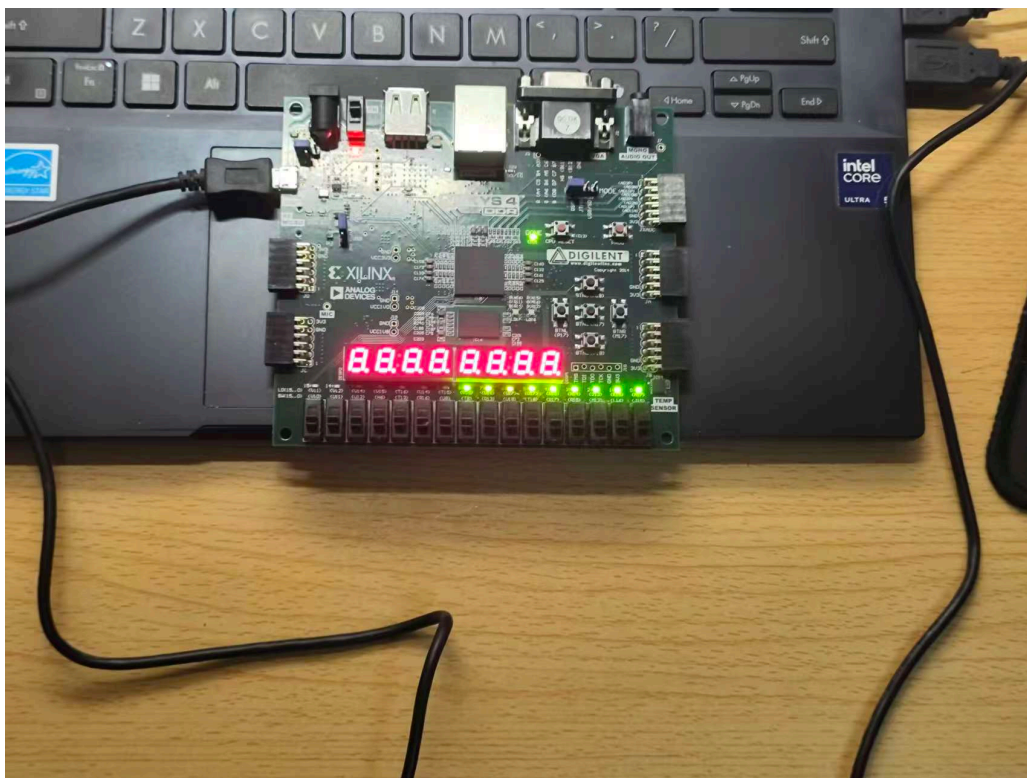


图 5: 调试 LED 运行状态——LD1/2/3 常亮

7.4 应用层验证：2048 与走迷宫

基础链路验证通过后，将第 6.4 节开发的应用固件烧录运行。

2048 串口原始数据。每次有效移动后，板载程序按协议输出结构化数据行，SSCOM 中可见形如 B:2,0,4,0,...|S:40|G:0 的文本（图 6）：B 段为 16 个格子数值，S 段为分数，G 段为游戏状态。



图 6: SSCOM 接收到的 2048 结构化数据

2048 图形界面。 PC 端 Python 程序解析上述数据并渲染为彩色棋盘，方块滑动与合并带有动画效果（图 7）。

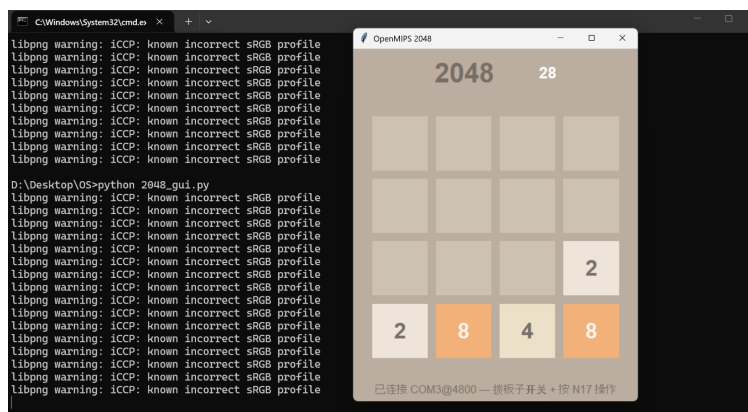


图 7: 2048 PC 端图形界面

走迷宫图形界面。 迷宫界面渲染随机生成的 8×8 完美迷宫，黄色为玩家、绿色为终点，玩家移动带有平滑动画（图 8）。多次复位可观察到不同布局的迷宫，验证了随机生成算法的有效性。



图 8: 走迷宫 PC 端图形界面

综合基础串口日志、数码管动态递增、调试 LED 组合状态，以及 2048 与走迷宫两款应用的端到端运行，确认 $\mu\text{C}/\text{OS-II}$ 实时操作系统已在 OpenMIPS + Nexys4 DDR 定制 SoC 平台上稳定运行，并具备承载交互式图形应用的能力。

8 总结

本实验从一款仅能访问片内 Block RAM 的独立 CPU 出发，完成了以下工作：引入 Wishbone 共享总线并挂载四台从设备，搭建功能完整的 SoC 硬件平台；修改 $\mu\text{C}/\text{OS-II}$ 体系结构相关代码，编写 BootLoader 完成 Flash 至 DDR2 的映像搬运与启动跳转；利用 mips-sde-elf 工具链完成交叉编译与二进制映像生成；在 Nexys4 DDR 开发板上完成全系统功能验证；并进一步在操作系统之上开发了 2048 与走迷宫两款交互式应用及 PC 端图形界面。各层面的工作内容与关键技术见表 9。

表 9: 实验工作内容汇总

层面	完成的工作	关键技术
时钟与复位	100 MHz → 50/200 MHz 双时钟域	Clocking Wizard、PLL 锁定接入复位
片上总线	Wishbone 共享总线挂载 4 从设备	WB_CONMAX、地址译码、握手协议
程序存储	SPI Flash 控制器（多状态 FSM）	STARTUPE2、SPI READ 时序、ILA 调试
运行内存	DDR2 子系统（MIG + Ram2Ddr）	MIG 配置、Wishbone-MIG 桥接
串口通信	UART16550（4800 bps / 8N1）	波特率分频、中断汇聚
通用 I/O	GPIO + 9 颗调试 LED	数码管段码驱动、LED 状态编码
操作系统	μC/OS-II 移植与交叉编译	上下文切换、栈帧初始化、GCC 工具链
应用开发	2048、走迷宫及 PC 图形界面	结构化协议、DFS 迷宫生成、tkinter 动画
系统验证	下板部署与全链路验证	Flash 烧录、串口捕获、实物与界面观测

在硬件设计方面，主要收获包括：掌握了 Wishbone 总线的地址译码与握手时序；实践了通过 STARTUPE2 原语驱动专用配置引脚的方法；熟悉了 MIG IP 核配置流程及 Wishbone-MIG 协议转换；并通过 Vivado ILA 完成了一次完整的时序调试闭环。在软件方面，完整经历了 BootLoader 编写、μC/OS-II 适配、交叉编译到映像烧录的端到端流程。在系统集成方面，最深刻的体会是：各模块独立测试正常并不保证系统联调一次通过——模块接口之间的时序偏差、启动顺序的隐性依赖和跨层信号的协议匹配，是系统集成阶段最频繁的故障来源。本实验中 SPI Flash 采样相位偏差便是一例：指令集正确、总线协议正确、Flash 芯片本身正常，三者之间的时序对齐关系出现单周期错位，即导致全系统无法工作。

应用开发部分带来了“软硬件协同”的直观体会：将板载程序定位为纯逻辑引擎、以结构化文本协议向 PC 推送状态、再由 PC 承担全部图形表现，使两端各自演进互不干扰——同一份固件既能被 SSCOM 以原始数据查看，也能被图形界面渲染为富表现力的交互体验。这种“瘦终端 + 富前端”的解耦，与嵌入式领域广泛采用的通信协议设计思路一脉相承。应用开发过程中亦遇到若干工程性问题：32 位交叉工具链在 64 位 WSL 下需要 libc6-i386 兼容库，且 drvfs 的大 inode 会导致 32 位编译器 stat 失败，需将源码置于 WSL 原生文件系统编译；此外还排查定位了开发板拨码开关接触不良等硬件偶发问题。这些经历使我对“软硬件一体化调试”有了更完整的认识。

全部实验目标均已达成，μC/OS-II 在自研 OpenMIPS SoC 上稳定运行，并成功承载了 2048 与走迷宫两款交互式图形应用。

A 关键模块源代码

A.1 SoC 顶层模块

```

1  `timescale 1ns / 1ps
2
3  //////////////////////////////////////////////////
4  // Module:  openmips_min_sopc
5  // Description: 最小 SoC 系统顶层 — OpenMIPS + 外设 + 总线互联
6  //           时钟: clk_in (100MHz 晶振) → clk_wiz → clk (CPU) +
           clk_200MHz (DDR2)
7  //           复位: rst_n (低有效) & locked (MMCM 锁定)
8  //           系统组成:
9  //               - openmips0:      OpenMIPS 处理器核心
10 //               - wb_conmax_top0:  Wishbone 互联矩阵 (8主×16从)
11 //               - uart_top0:      UART 16550 串口 (从设备 1)
12 //               - gpio_top0:      GPIO 控制器 (从设备 2)
13 //               - flash_rom:      SPI Flash ROM (从设备 3, 存放指令)
14 //               - DDR2:           DDR2 SDRAM 控制器 (从设备 0)
15 //           中断: {3'b0, gpio_int, uart_int, timer_int} → CPU int_i
16 //           Debug LED: heartbeat + CPU状态 + DDR2状态 + UART TX
17 //           目标板: Nexys4 DDR (Xilinx Artix-7)
18  //////////////////////////////////////////////////
19
20  `include "defines.v"
21
22  module openmips_min_sopc(
23      input wire clk_in,
24      input wire rst_n,
25      input wire flash_continue,
26
27      // Debug LEDs
28      output wire [8:0] debug_led,
29
30      // UART
31      input wire      uart_in,
32      output wire      uart_out,
33
34      // GPIO
35      input wire [15:0] gpio_i,
36      output wire [7:0] gpio_o, //31:16-led, 15:8-o_seg, 7:0-o_sel
37
38      // Flash
39      output cs_n,

```

```

40     input    sdi,
41     output   sdo,
42     output   wp_n,
43     output   hld_n,
44
45     // DDR2
46     inout [15:0]      ddr2_dq,
47     inout [1:0]       ddr2_dqs_n,
48     inout [1:0]       ddr2_dqs_p,
49     output [12:0]      ddr2_addr,
50     output [2:0]       ddr2_ba,
51     output             ddr2_ras_n,
52     output             ddr2_cas_n,
53     output             ddr2_we_n,
54     output [0:0]       ddr2_ck_p,
55     output [0:0]       ddr2_ck_n,
56     output [0:0]       ddr2_cke,
57     output [0:0]       ddr2_cs_n,
58     output [1:0]       ddr2_dm,
59     output [0:0]       ddr2_odt
60 );
61
62     wire[31:0]  m0_data_i,m0_data_o,m0_addr_i;
63     wire[3:0]   m0_sel_i;
64     wire        m0_we_i,m0_cyc_i,m0_stb_i,m0_ack_o;
65
66     wire[31:0]  m1_data_i,m1_data_o,m1_addr_i;
67     wire[3:0]   m1_sel_i;
68     wire        m1_we_i,m1_cyc_i,m1_stb_i,m1_ack_o;
69
70     wire[31:0]  s0_data_i,s0_data_o,s0_addr_o;
71     wire[3:0]   s0_sel_o;
72     wire        s0_we_o,s0_cyc_o,s0_stb_o,s0_ack_i;
73
74     wire[31:0]  s1_data_i,s1_data_o,s1_addr_o;
75     wire[3:0]   s1_sel_o;
76     wire        s1_we_o,s1_cyc_o,s1_stb_o,s1_ack_i;
77
78     wire[31:0]  s2_data_i,s2_data_o,s2_addr_o;
79     wire[3:0]   s2_sel_o;
80     wire        s2_we_o,s2_cyc_o,s2_stb_o,s2_ack_i;
81
82     wire[31:0]  s3_data_i,s3_data_o,s3_addr_o;
83     wire[3:0]   s3_sel_o;
84     wire        s3_we_o,s3_cyc_o,s3_stb_o,s3_ack_i;

```

```

85
86 //#####
87     wire clk;
88     wire clk_200MHz;
89     wire locked;
90     wire rst;
91     assign rst = ~rst_n | ~locked;
92
93     clk_wiz_0 clk_wiz_inst(
94         .clk_in1(clk_in),
95         .clk_out1(clk),
96         .clk_out2(clk_200MHz),
97         .locked(locked)
98     );
99
100 //#####
101     wire [5:0] int;
102     wire timer_int,gpio_int,uart_int;
103     assign int = {3'b000,gpio_int,uart_int,timer_int};
104
105     openmips openmips0(
106         .clk(clk),          .rst(rst),
107
108         .iwishbone_data_i(m1_data_o),.iwishbone_ack_i(m1_ack_o),
109         .iwishbone_addr_o(m1_addr_i),.iwishbone_data_o(m1_data_i),
110         .iwishbone_we_o(m1_we_i),.iwishbone_sel_o(m1_sel_i),
111         .iwishbone_stb_o(m1_stb_i),.iwishbone_cyc_o(m1_cyc_i),
112
113         .int_i(int),
114
115         .dwishbone_data_i(m0_data_o),.dwishbone_ack_i(m0_ack_o),
116         .dwishbone_addr_o(m0_addr_i),.dwishbone_data_o(m0_data_i),
117         .dwishbone_we_o(m0_we_i),.dwishbone_sel_o(m0_sel_i),
118         .dwishbone_stb_o(m0_stb_i),.dwishbone_cyc_o(m0_cyc_i),
119
120         .timer_int_o(timer_int)
121     );
122
123 //#####
124     uart_top uart_top0(
125         .wb_clk_i(clk),.wb_rst_i(rst),
126         .wb_adr_i(s1_addr_o[4:0]),.wb_dat_i(s1_data_o),
127         .wb_dat_o(s1_data_i),.wb_we_i(s1_we_o),
128         .wb_stb_i(s1_stb_o),.wb_cyc_i(s1_cyc_o),
129         .wb_ack_o(s1_ack_i),.wb_sel_i(s1_sel_o),

```

```

130
131     .int_o(uart_int),
132
133     .stx_pad_o(uart_out),.srx_pad_i(uart_in),
134     .cts_pad_i(1'b0),.dsr_pad_i(1'b0),
135     .ri_pad_i(1'b0),.dcd_pad_i(1'b0),
136     .rts_pad_o(),.dtr_pad_o()
137 );
138
139 //#####
140 //     wire[31:0] gpio_o;
141 wire sdram_init_done;
142 wire [31:0] gpio_i_temp;
143 assign gpio_i_temp={15'h0,1'b1,gpio_i}; // bypass DDR2 init wait
144 wire [31:0] gpio_o_temp;
145 assign gpio_o[7:0] = gpio_o_temp[7:0];
146
147 gpio_top gpio_top0(
148     .wb_clk_i(clk),.wb_rst_i(rst),
149     .wb_cyc_i(s2_cyc_o),.wb_adr_i(s2_addr_o[7:0]),
150     .wb_dat_i(s2_data_o),.wb_sel_i(s2_sel_o),
151     .wb_we_i(s2_we_o),.wb_stb_i(s2_stb_o),
152     .wb_dat_o(s2_data_i),.wb_ack_o(s2_ack_i),
153     .wb_err_o(),
154
155     .wb_inta_o(gpio_int),
156     .ext_pad_i(gpio_i_temp),
157     .ext_pad_o(gpio_o_temp),
158     .ext_padoe_o()
159 );
160
161 //#####
162 flash_rom flash_rom(
163     .wb_clk_i(clk), //100MHz
164     .wb_rst_i(rst),
165     .wb_adr_i({s3_addr_o[23:2],2'b00}),
166     .wb_dat_o(s3_data_i),
167     .wb_dat_i(s3_data_o),
168     .wb_sel_i(s3_sel_o),
169     .wb_we_i(s3_we_o),
170     .wb_stb_i(s3_stb_o),
171     .wb_cyc_i(s3_cyc_o),
172     .wb_ack_o(s3_ack_i),
173
174     .flash_continue(flash_continue),

```



```

175     .cs_n(cs_n),
176     .sdi(sdi),
177     .sdo(sdo),
178     .wp_n(wp_n),
179     .hld_n(hld_n)
180 );
181
182 //#####
183 DDR2 DDR2(
184     .wb_rst_i(1'b0),
185     .wb_clk_i(clk),
186     .clk_200MHz_i(clk_200MHz),
187
188     .wb_stb_i(s0_stb_o),
189     .wb_ack_o(s0_ack_i),
190     .wb_adr_i({s0_addr_o[26:2],2'b00}),
191     .wb_we_i(s0_we_o),
192     .wb_dat_i(s0_data_o),
193     .wb_sel_i(s0_sel_o),
194     .wb_dat_o(s0_data_i),
195     .wb_cyc_i(s0_cyc_o),
196
197     .init_calib_complete(sdram_init_done),
198
199     .ddr2_ck_p(ddr2_ck_p),
200     .ddr2_ck_n(ddr2_ck_n),
201     .ddr2_cke(ddr2_cke),
202     .ddr2_cs_n(ddr2_cs_n),
203     .ddr2_ras_n(ddr2_ras_n),
204     .ddr2_cas_n(ddr2_cas_n),
205     .ddr2_we_n(ddr2_we_n),
206     .ddr2_dm(ddr2_dm),
207     .ddr2_ba(ddr2_ba),
208     .ddr2_addr(ddr2_addr),
209     .ddr2_dq(ddr2_dq),
210     .ddr2_dqs_p(ddr2_dqs_p),
211     .ddr2_dqs_n(ddr2_dqs_n),
212     .ddr2_odt(ddr2_odt)
213 );
214
215 //#####
216 wb_conmax_top wb_conmax_top0(
217     .clk_i(clk),.rst_i(rst),
218
219     .m0_data_i(m0_data_i),.m0_data_o(m0_data_o),

```

```
220     .m0_addr_i(m0_addr_i),.m0_sel_i(m0_sel_i),
221     .m0_we_i(m0_we_i),.m0_cyc_i(m0_cyc_i),
222     .m0_stb_i(m0_stb_i),.m0_ack_o(m0_ack_o),
223
224     .m1_data_i(m1_data_i),.m1_data_o(m1_data_o),
225     .m1_addr_i(m1_addr_i),.m1_sel_i(m1_sel_i),
226     .m1_we_i(m1_we_i),.m1_cyc_i(m1_cyc_i),
227     .m1_stb_i(m1_stb_i),.m1_ack_o(m1_ack_o),
228
229     .m2_data_i(`ZeroWord),.m2_data_o(),
230     .m2_addr_i(`ZeroWord),.m2_sel_i(4'b0000),
231     .m2_we_i(1'b0),.m2_cyc_i(1'b0),
232     .m2_stb_i(1'b0),.m2_ack_o(),
233     .m2_err_o(),.m2_rty_o(),
234
235     .m3_data_i(`ZeroWord),.m3_data_o(),
236     .m3_addr_i(`ZeroWord),.m3_sel_i(4'b0000),
237     .m3_we_i(1'b0),.m3_cyc_i(1'b0),
238     .m3_stb_i(1'b0),.m3_ack_o(),
239     .m3_err_o(),.m3_rty_o(),
240
241     .m4_data_i(`ZeroWord),.m4_data_o(),
242     .m4_addr_i(`ZeroWord),.m4_sel_i(4'b0000),
243     .m4_we_i(1'b0),.m4_cyc_i(1'b0),
244     .m4_stb_i(1'b0),.m4_ack_o(),
245     .m4_err_o(),.m4_rty_o(),
246
247     .m5_data_i(`ZeroWord),.m5_data_o(),
248     .m5_addr_i(`ZeroWord),.m5_sel_i(4'b0000),
249     .m5_we_i(1'b0),.m5_cyc_i(1'b0),
250     .m5_stb_i(1'b0),.m5_ack_o(),
251     .m5_err_o(),.m5_rty_o(),
252
253     .m6_data_i(`ZeroWord),.m6_data_o(),
254     .m6_addr_i(`ZeroWord),.m6_sel_i(4'b0000),
255     .m6_we_i(1'b0),.m6_cyc_i(1'b0),
256     .m6_stb_i(1'b0),.m6_ack_o(),
257     .m6_err_o(),.m6_rty_o(),
258
259     .m7_data_i(`ZeroWord),.m7_data_o(),
260     .m7_addr_i(`ZeroWord),.m7_sel_i(4'b0000),
261     .m7_we_i(1'b0),.m7_cyc_i(1'b0),
262     .m7_stb_i(1'b0),.m7_ack_o(),
263     .m7_err_o(),.m7_rty_o(),
264
```

```
265     .s0_data_i(s0_data_i),.s0_data_o(s0_data_o),
266     .s0_addr_o(s0_addr_o),.s0_sel_o(s0_sel_o),
267     .s0_we_o(s0_we_o),.s0_cyc_o(s0_cyc_o),
268     .s0_stb_o(s0_stb_o),.s0_ack_i(s0_ack_i),
269     .s0_err_i(1'b0),.s0_rty_i(1'b0),
270
271     .s1_data_i(s1_data_i),.s1_data_o(s1_data_o),
272     .s1_addr_o(s1_addr_o),.s1_sel_o(s1_sel_o),
273     .s1_we_o(s1_we_o),.s1_cyc_o(s1_cyc_o),
274     .s1_stb_o(s1_stb_o),.s1_ack_i(s1_ack_i),
275     .s1_err_i(1'b0),.s1_rty_i(1'b0),
276
277     .s2_data_i(s2_data_i),.s2_data_o(s2_data_o),
278     .s2_addr_o(s2_addr_o),.s2_sel_o(s2_sel_o),
279     .s2_we_o(s2_we_o),.s2_cyc_o(s2_cyc_o),
280     .s2_stb_o(s2_stb_o),.s2_ack_i(s2_ack_i),
281     .s2_err_i(1'b0),.s2_rty_i(1'b0),
282
283     .s3_data_i(s3_data_i),.s3_data_o(s3_data_o),
284     .s3_addr_o(s3_addr_o),.s3_sel_o(s3_sel_o),
285     .s3_we_o(s3_we_o),.s3_cyc_o(s3_cyc_o),
286     .s3_stb_o(s3_stb_o),.s3_ack_i(s3_ack_i),
287     .s3_err_i(1'b0),.s3_rty_i(1'b0),
288
289     .s4_data_i(),.s4_data_o(),
290     .s4_addr_o(),.s4_sel_o(),
291     .s4_we_o(),.s4_cyc_o(),
292     .s4_stb_o(),.s4_ack_i(1'b0),
293     .s4_err_i(1'b0),.s4_rty_i(1'b0),
294
295     .s5_data_i(),.s5_data_o(),
296     .s5_addr_o(),.s5_sel_o(),
297     .s5_we_o(),.s5_cyc_o(),
298     .s5_stb_o(),.s5_ack_i(1'b0),
299     .s5_err_i(1'b0),.s5_rty_i(1'b0),
300
301     .s6_data_i(),.s6_data_o(),
302     .s6_addr_o(),.s6_sel_o(),
303     .s6_we_o(),.s6_cyc_o(),
304     .s6_stb_o(),.s6_ack_i(1'b0),
305     .s6_err_i(1'b0),.s6_rty_i(1'b0),
306
307     .s7_data_i(),.s7_data_o(),
308     .s7_addr_o(),.s7_sel_o(),
309     .s7_we_o(),.s7_cyc_o(),
```

```
310     .s7_stb_o(),.s7_ack_i(1'b0),
311     .s7_err_i(1'b0),.s7_rty_i(1'b0),
312
313     .s8_data_i(),.s8_data_o(),
314     .s8_addr_o(),.s8_sel_o(),
315     .s8_we_o(),.s8_cyc_o(),
316     .s8_stb_o(),.s8_ack_i(1'b0),
317     .s8_err_i(1'b0),.s8_rty_i(1'b0),
318
319     .s9_data_i(),.s9_data_o(),
320     .s9_addr_o(),.s9_sel_o(),
321     .s9_we_o(),.s9_cyc_o(),
322     .s9_stb_o(),.s9_ack_i(1'b0),
323     .s9_err_i(1'b0),.s9_rty_i(1'b0),
324
325     .s10_data_i(),.s10_data_o(),
326     .s10_addr_o(),.s10_sel_o(),
327     .s10_we_o(),.s10_cyc_o(),
328     .s10_stb_o(),.s10_ack_i(1'b0),
329     .s10_err_i(1'b0),.s10_rty_i(1'b0),
330
331     .s11_data_i(),.s11_data_o(),
332     .s11_addr_o(),.s11_sel_o(),
333     .s11_we_o(),.s11_cyc_o(),
334     .s11_stb_o(),.s11_ack_i(1'b0),
335     .s11_err_i(1'b0),.s11_rty_i(1'b0),
336
337     .s12_data_i(),.s12_data_o(),
338     .s12_addr_o(),.s12_sel_o(),
339     .s12_we_o(),.s12_cyc_o(),
340     .s12_stb_o(),.s12_ack_i(1'b0),
341     .s12_err_i(1'b0),.s12_rty_i(1'b0),
342
343     .s13_data_i(),.s13_data_o(),
344     .s13_addr_o(),.s13_sel_o(),
345     .s13_we_o(),.s13_cyc_o(),
346     .s13_stb_o(),.s13_ack_i(1'b0),
347     .s13_err_i(1'b0),.s13_rty_i(1'b0),
348
349     .s14_data_i(),.s14_data_o(),
350     .s14_addr_o(),.s14_sel_o(),
351     .s14_we_o(),.s14_cyc_o(),
352     .s14_stb_o(),.s14_ack_i(1'b0),
353     .s14_err_i(1'b0),.s14_rty_i(1'b0),
354
```

```

355     .s15_data_i(), .s15_data_o(),
356     .s15_addr_o(), .s15_sel_o(),
357     .s15_we_o(), .s15_cyc_o(),
358     .s15_stb_o(), .s15_ack_i(1'b0),
359     .s15_err_i(1'b0), .s15_rty_i(1'b0)
360 );
361
362 //#####
363 // Debug LEDs: runtime status indicators (ILA kept separately for debug)
364 reg [25:0] heartbeat_cnt;
365 always @(posedge clk or posedge rst) begin
366     if (rst) heartbeat_cnt <= 26'd0;
367     else      heartbeat_cnt <= heartbeat_cnt + 26'd1;
368 end
369
370 reg cpu_fetch_active;    // CPU has fetched at least once from Flash
371 reg uart_accessed;      // CPU has accessed the UART
372 always @(posedge clk or posedge rst) begin
373     if (rst) begin
374         cpu_fetch_active <= 1'b0;
375         uart_accessed    <= 1'b0;
376     end else begin
377         if (s3_cyc_o & s3_stb_o) cpu_fetch_active <= 1'b1;
378         if (s1_cyc_o & s1_stb_o) uart_accessed    <= 1'b1;
379     end
380 end
381
382 assign debug_led[0] = heartbeat_cnt[25];    // ~0.75Hz heartbeat: system
alive
383 assign debug_led[1] = cpu_fetch_active;    // CPU fetching from Flash
384 assign debug_led[2] = uart_accessed;      // CPU has talked to UART
385 assign debug_led[3] = sdram_init_done;    // DDR2 calibration done
386 assign debug_led[4] = heartbeat_cnt[24];  // marquee: visual running
indicator
387 assign debug_led[5] = heartbeat_cnt[23];
388 assign debug_led[6] = heartbeat_cnt[22];
389 assign debug_led[7] = heartbeat_cnt[21];
390 assign debug_led[8] = uart_out;           // live UART TX line: blinks
while sending
391
392 endmodule

```

Listing 4: openmips_min_soc.v

A.2 SPI Flash 控制器

```

1 `timescale 1ns / 1ps
2
3 //////////////////////////////////////////////////
4 // Module:  flash_rom
5 // Description: SPI Flash ROM 控制器 — Wishbone 从设备接口
6 //          两大功能:
7 //          1. SPI Flash 控制 — 通过 SPI 接口读写板载 Flash 存储器
8 //          2. Wishbone 总线从设备 — 响应 CPU 取指/数据访问
9 //          Flash 作为 CPU 的指令存储器 (inst_rom)
10 //          内部状态机: IDLE → START → INST_OUT → ADDR1/2/3_OUT
11 //          → WRITE_DATA 或 READ_DATA(1~5) → WAITING →
12 //          ENDING
13 //          支持连续读模式 (flash_continue 信号)
14 //          注: 复位后需等待 Flash 初始化完成才能开始取指
15 //////////////////////////////////////////////////
16 // main two part: 1.spi flash control 2. wb main bus
17 module flash_rom(
18     input wire wb_clk_i, // Wishbone 时钟
19     input wire wb_rst_i, // 复位
20     input wire wb_cyc_i, // 周期信号
21     input wire wb_stb_i, // 片选
22     input wire wb_we_i, // 写使能
23     input wire [3:0] wb_sel_i, // 4-bit 片选
24     input wire [23:0] wb_adr_i, // 24-bit 地址
25     input wire [31:0] wb_dat_i, // 32-bit 数据
26
27     output reg [31:0] wb_dat_o, // 32-bit 数据
28     output reg        wb_ack_o, // 响应
29
30     input wire flash_continue, // 连续读模式
31
32     output reg cs_n, // 片选
33     input  sdi, // 串行数据输入
34     output reg sdo, // 串行数据输出
35     output reg wp_n, // 写使能
36     output reg hld_n, // 保持
37 );
38
39
40 parameter IDLE      = 5'b00000;
41 parameter START     = 5'b00010;

```

```

42 parameter INST_OUT    = 5'b00011;
43 parameter ADDR1_OUT   = 5'b00100;
44 parameter ADDR2_OUT   = 5'b00101;
45 parameter ADDR3_OUT   = 5'b00110;
46 parameter WRITE_DATA  = 5'b00111;
47 parameter READ_DATA   = 5'b01000;
48 parameter READ_DATA1=5'b01001;
49 parameter READ_DATA2=5'b01010;
50 parameter READ_DATA3=5'b01011;
51 parameter READ_DATA4=5'b01100;
52 parameter READ_DATA5=5'b01101;
53 parameter WAITING     =5'b10000;
54 parameter ENDING      = 5'b10001;
55
56 (* dont_touch = "true" *)reg[4:0] init_count;
57
58 (* dont_touch = "true", mark_debug = "true" *)reg          sck;
59 (* dont_touch = "true", mark_debug = "true" *)reg [4:0] state;
60 reg [4:0] next_state;
61
62 (* dont_touch = "true" *)reg [7:0] instruction;
63 (* dont_touch = "true", mark_debug = "true" *)reg [7:0] datain_shift;
64 (* dont_touch = "true" *)reg [7:0] datain;
65 (* dont_touch = "true" *)reg [7:0] dataout;
66 (* dont_touch = "true" *)reg          sck_en;
67 (* dont_touch = "true" *)reg [2:0] sck_en_d;
68 (* dont_touch = "true" *)reg [10:0] read_count;
69 reg [2:0] cs_n_d;
70
71 reg          temp;
72 (* dont_touch = "true", mark_debug = "true" *)reg [3:0] sdo_count;
73 reg [15:0] page_count;
74 reg [7:0] wait_count;
75 (* dont_touch = "true" *)reg [23:0] addr;
76 reg          wrh_rdl; // High indicates write, low indicates read
77 reg          addr_req; // Address writing requested
78 reg [15:0] wr_cnt; // Number of bytes to be written
79 reg [15:0] rd_cnt; // Number of bytes to be read
80 (* dont_touch = "true", mark_debug = "true" *)reg [31:0] read_data;
81
82 // ILA probes: sdi/sdo/cs_n are ports, expose as marked nets so Set Up
   Debug lists them
83 (* dont_touch = "true", mark_debug = "true" *) wire sdi_dbg;
84 (* dont_touch = "true", mark_debug = "true" *) wire sdo_dbg;
85 (* dont_touch = "true", mark_debug = "true" *) wire cs_n_dbg;

```

```
86 assign sdi_dbg = sdi;
87 assign sdo_dbg = sdo;
88 assign cs_n_dbg = cs_n;
89
90
91
92 // State machine
93 always @(posedge wb_clk_i or posedge wb_rst_i) begin
94     if(wb_rst_i) begin
95         state <= IDLE;
96         read_count <= 11'd0;
97         wb_ack_o <= 1'b0;
98         init_count <= 5'd2;
99     end
100     else if(wb_cyc_i & wb_stb_i) begin
101         state <= next_state;
102         if(state == ENDING && !wb_ack_o)
103             begin
104                 if(init_count > 5'd0)
105                     begin
106                         init_count <= init_count - 5'd1;
107                         state <= IDLE;
108                     end
109                 else
110                     begin
111                         wb_ack_o <= 1'b1;
112                     end
113             end
114     end
115     else begin
116         state <= IDLE;
117         wb_ack_o <= 1'b0;
118     end
119 end
120
121 always @(posedge wb_clk_i)
122 begin
123     //wb_dat_o <= read_data;
124     wp_n <= 1'b1;
125     hld_n <= 1'b1;
126 end
127
128 always @(posedge wb_clk_i or posedge wb_rst_i) begin
129     if(wb_rst_i) begin
130         next_state <= IDLE;
```



```

131      sck_en      <= 1'b0;
132      cs_n_d[0]   <= 1'b1;
133      dataout     <= 8'd0;
134      sdo_count   <= 4'd0;
135      sdo         <= 1'b0;
136      datain      <= 8'd0;
137      addr        <=24'd0;
138      datain_shift<=8'd0;
139
140      temp        <= 1'b0;
141      page_count  <= 16'd0;
142      wait_count  <= 8'd0;
143      read_data   <=32'd0;
144
145  end
146  else begin
147      case(state)
148      IDLE:
149      begin // IDLE state
150          wait_count <= 8'd0;
151          next_state<=START;
152      end
153
154      START:
155      begin // enable SCK and CS
156          addr<=wb_adr_i;
157          //addr<=24'd0;
158          sck_en <= 1'b1;
159          cs_n_d[0] <= 1'b0;
160          next_state <= INST_OUT;
161          read_count<=read_count+11'd1;
162      end
163      INST_OUT:
164      begin // send out instruction
165          if(sdo_count == 4'd1) begin
166              {sdo, dataout[6:0]} <= instruction;
167          end
168          else if(sdo_count[0]) begin
169              {sdo, dataout[6:0]} <= {dataout[6:0],1'b0};
170          end
171
172          if(sdo_count != 4'd15) begin
173              sdo_count <= sdo_count + 4'd1;
174          end
175          else begin

```

```

176         sdo_count  <= 4'd0;
177         next_state <= (addr_req) ?  ADDR1_OUT : ((wrh_rdl) ? ((
wr_cnt==16'd0) ? ENDING : WRITE_DATA) : ((rd_cnt==16'd0) ? ENDING :
READ_DATA1));
178     end
179 end
180 ADDR1_OUT:
181 begin    // send out address[23:16]
182     if(sdo_count == 4'd1) begin
183         {sdo, dataout[6:0]} <= addr[23:16];
184     end
185     else if(sdo_count[0]) begin
186         {sdo, dataout[6:0]} <= {dataout[6:0],1'b0};
187     end
188
189     if(sdo_count != 4'd15) begin
190         sdo_count <= sdo_count + 4'd1;
191     end
192     else begin
193         sdo_count  <= 4'd0;
194         next_state <= ADDR2_OUT;
195     end
196 end
197 ADDR2_OUT:
198 begin    // send out address[15:8]
199     if(sdo_count == 4'd1) begin
200         {sdo, dataout[6:0]} <= addr[15:8];
201     end
202     else if(sdo_count[0]) begin
203         {sdo, dataout[6:0]} <= {dataout[6:0],1'b0};
204     end
205
206     if(sdo_count != 4'd15) begin
207         sdo_count <= sdo_count + 4'd1;
208     end
209     else begin
210         sdo_count  <= 4'd0;
211         next_state <= ADDR3_OUT;
212     end
213 end
214 ADDR3_OUT:
215 begin    // send out address[7:0]
216     if(sdo_count == 4'd1) begin
217         {sdo, dataout[6:0]} <= addr[7:0];
218     end

```

```

219         else if(sdo_count[0]) begin
220             {sdo, dataout[6:0]} <= {dataout[6:0],1'b0};
221         end
222
223         if(sdo_count != 4'd15) begin
224             sdo_count <= sdo_count + 4'd1;
225         end
226         else begin
227             sdo_count <= 4'd0;
228             next_state <= (wrh_rdl) ? ((wr_cnt==16'd0) ? ENDING :
WRITE_DATA) : ((rd_cnt==16'd0) ? ENDING : READ_DATA1);
229             page_count <= 16'd0;
230         end
231     end
232     WRITE_DATA:
233     begin // send testing data out to flash
234         if(sdo_count == 4'd1) begin
235             {sdo, dataout[6:0]} <= 8'h5A;
236         end
237         else if(sdo_count[0]) begin
238             {sdo, dataout[6:0]} <= {dataout[6:0],1'b0};
239         end
240
241         if(sdo_count != 4'd15) begin
242             sdo_count <= sdo_count + 4'd1;
243         end
244         else begin
245             page_count <= page_count + 16'd1;
246             sdo_count <= 4'd0;
247             next_state <= (page_count < (wr_cnt-16'd1)) ? WRITE_DATA :
ENDING;
248         end
249     end
250     READ_DATA1:
251     begin // get the first data from flash
252         if(sdo_count[0]) begin
253             datain_shift <= {datain_shift[6:0],sdi};
254         end
255
256         if(sdo_count == 4'd1) begin
257             datain <= {datain_shift, sdi};
258         end
259
260         if(sdo_count != 4'd15) begin
261             sdo_count <= sdo_count + 4'd1;

```

```
262     end
263     else begin
264         page_count <= page_count + 16'd1;
265         sdo_count <= 4'd0;
266         next_state <=READ_DATA2;
267     end
268 end
269 READ_DATA2:
270 begin // get the first data from flash
271     if(sdo_count[0]) begin
272         datain_shift <= {datain_shift[6:0],sdi};
273     end
274
275     if(sdo_count == 4'd2) begin
276         read_data[31:24] <= datain_shift;
277         datain<= datain_shift;
278     end
279
280     if(sdo_count != 4'd15) begin
281         sdo_count <= sdo_count + 4'd1;
282     end
283     else begin
284         page_count <= page_count + 16'd1;
285         sdo_count <= 4'd0;
286         next_state <=READ_DATA3;
287     end
288 end
289 READ_DATA3:
290 begin // get the first data from flash
291     if(sdo_count[0]) begin
292         datain_shift <= {datain_shift[6:0],sdi};
293     end
294
295     if(sdo_count == 4'd2) begin
296         read_data[23:16] <= datain_shift;
297         datain<= datain_shift;
298     end
299
300     if(sdo_count != 4'd15) begin
301         sdo_count <= sdo_count + 4'd1;
302     end
303     else begin
304         page_count <= page_count + 16'd1;
305         sdo_count <= 4'd0;
306         next_state <=READ_DATA4;
```

```

307         end
308     end
309     READ_DATA4:
310     begin    // get the first data from flash
311         if(sdo_count[0]) begin
312             datain_shift <= {datain_shift[6:0],sdi};
313         end
314
315         if(sdo_count == 4'd2) begin
316             read_data[15:8] <= datain_shift;
317             datain <= datain_shift;
318         end
319
320         if(sdo_count != 4'd15) begin
321             sdo_count <= sdo_count + 4'd1;
322         end
323         else begin
324             page_count <= page_count + 16'd1;
325             sdo_count <= 4'd0;
326             next_state <= READ_DATA5;
327         end
328     end
329     READ_DATA5:
330     begin    // get the first data from flash
331         if(sdo_count[0]) begin
332             datain_shift <= {datain_shift[6:0],sdi};
333         end
334
335         if(sdo_count == 4'd2) begin
336             read_data[7:0] <= datain_shift;
337             datain <= datain_shift;
338         end
339
340         if(sdo_count != 4'd15) begin
341             sdo_count <= sdo_count + 4'd1;
342         end
343         else begin
344             page_count <= page_count + 16'd1;
345             sdo_count <= 4'd0;
346
347             next_state <= WAITING;
348         end
349     end
350     WAITING:
351     begin    //disable SCK and CS, wait for 32 clock cycles

```

```

352         sck_en <= 1'b0;
353         cs_n_d[0] <= 1'b1;
354         sdo_count <= 4'd0;
355
356         next_state<=ENDING;
357
358     end
359     ENDING:
360     begin
361
362     end
363     endcase
364 end
365 end
366 // SCK generator, 50MHz output
367 always @(posedge wb_clk_i) begin
368     sck_en_d <= {sck_en_d[1:0],sck_en};
369 end
370
371 always @(posedge wb_clk_i or posedge wb_rst_i) begin
372     if(wb_rst_i) begin
373         sck <= 1'b0;
374     end
375     else if(sck_en_d[2] & sck_en) begin
376         sck <= ~sck;
377     end
378     else begin
379         sck <= 1'b0;
380     end
381 end
382
383 always @(posedge wb_clk_i or posedge wb_rst_i) begin
384     if(wb_rst_i) begin
385         {cs_n,cs_n_d[2:1]} <= 3'h7;
386     end
387     else begin
388         {cs_n,cs_n_d[2:1]} <= cs_n_d;
389     end
390 end
391
392 STARTUPE2
393 #(
394     .PROG_USR("FALSE"),
395     .SIM_CCLK_FREQ(10.0)
396 )

```

```

397 STARTUPE2_inst
398 (
399     .CFGCLK      (),
400     .CFGMCLK     (),
401     .EOS         (),
402     .PREQ        (),
403     .CLK         (1'b0),
404     .GSR         (1'b0),
405     .GTS         (1'b0),
406     .KEYCLEARB   (1'b0),
407     .PACK        (1'b0),
408     .USRCCLKO    (sck),      // First three cycles after config ignored, see
                                AR# 52626
409     .USRCCLKTS   (1'b0),      // 0 to enable CCLK output
410     .USRDONEO    (1'b1),      // Shouldn't matter if tristate is high, but
                                generates a warning if tied low.
411     .USRDONETS   (1'b1)      // 1 to tristate DONE output
412 );
413
414 // ROM for instructions
415 always @(posedge wb_clk_i) begin
416     instruction <= 8'h03; wrh_rdl <= 1'b0; addr_req <= 1'b1;
417     wr_cnt <= 16'd0; rd_cnt <= 16'd4;    // READ
418
419 end
420
421 always@(posedge wb_clk_i)
422     begin
423         // for real borad
424         wb_dat_o <= read_data;
425     end
426
427 endmodule

```

Listing 5: flash_rom.v

A.3 Wishbone 总线接口

```

1 ///////////////////////////////////////////////////////////////////
2 // Module:  wishbone_bus_if
3 // File:    wishbone_bus_if.v
4 // Description: Wishbone 总线接口 — CPU 侧读写操作到 Wishbone 总线的桥接
5 //              状态机: WB_IDLE → WB_BUSY → WB_WAIT_FOR_STALL → WB_IDLE
6 //              - WB_IDLE: 空闲, 等待 CPU 发起访问 (cpu_ce_i)

```

```

7 //          - WB_BUSY: 总线事务进行中, 等待 ack 或 flush
8 //          - WB_WAIT_FOR_STALL: 事务完成但流水线暂停, 等待 stall 释放
9 //          支持 stall/flush 控制, 与 CPU 流水线同步
10 //          在 openmips.v 中实例化两次:
11 //          - iwishbone_bus_if: 取指 Wishbone 接口
12 //          - dwishbone_bus_if: 数据访存 Wishbone 接口
13 ///////////////////////////////////////////////////
14
15 `include "defines.v"
16
17 module wishbone_bus_if(
18
19     input  wire                                clk,
20     input wire                                rst,
21
22     //??????ctrl
23     input wire[5:0]                          stall_i,
24     input                                flush_i,
25
26     //CPU???????
27     input wire                                cpu_ce_i,
28     input wire[`RegBus]                      cpu_data_i,
29     input wire[`RegBus]                      cpu_addr_i,
30     input wire                                cpu_we_i,
31     input wire[3:0]                          cpu_sel_i,
32     output reg[`RegBus]                      cpu_data_o,
33
34     //Wishbone?????
35     input wire[`RegBus]                      wishbone_data_i,
36     input wire                                wishbone_ack_i,
37     output reg[`RegBus]                      wishbone_addr_o,
38     output reg[`RegBus]                      wishbone_data_o,
39     output reg                                wishbone_we_o,
40     output reg[3:0]                          wishbone_sel_o,
41     output reg                                wishbone_stb_o,
42     output reg                                wishbone_cyc_o,
43
44     output reg                                stallreq
45
46 );
47
48 reg[1:0] wishbone_state;
49 reg[`RegBus] rd_buf;
50
51 always @ (posedge clk) begin

```



```

52     if(rst == `RstEnable) begin
53         wishbone_state <= `WB_IDLE;
54         wishbone_addr_o <= `ZeroWord;
55         wishbone_data_o <= `ZeroWord;
56         wishbone_we_o <= `WriteDisable;
57         wishbone_sel_o <= 4'b0000;
58         wishbone_stb_o <= 1'b0;
59         wishbone_cyc_o <= 1'b0;
60         rd_buf <= `ZeroWord;
61 //         cpu_data_o <= `ZeroWord;
62     end else begin
63         case (wishbone_state)
64             `WB_IDLE:         begin
65                 if((cpu_ce_i == 1'b1) && (flush_i == `False_v)) begin
66                     wishbone_stb_o <= 1'b1;
67                     wishbone_cyc_o <= 1'b1;
68                     wishbone_addr_o <= cpu_addr_i;
69                     wishbone_data_o <= cpu_data_i;
70                     wishbone_we_o <= cpu_we_i;
71                     wishbone_sel_o <=  cpu_sel_i;
72                     wishbone_state <= `WB_BUSY;
73                     rd_buf <= `ZeroWord;
74 //                 end else begin
75 //                     wishbone_state <= WB_IDLE;
76 //                     wishbone_addr_o <= `ZeroWord;
77 //                     wishbone_data_o <= `ZeroWord;
78 //                     wishbone_we_o <= `WriteDisable;
79 //                     wishbone_sel_o <= 4'b0000;
80 //                     wishbone_stb_o <= 1'b0;
81 //                     wishbone_cyc_o <= 1'b0;
82 //                     cpu_data_o <= `ZeroWord;
83                 end
84             end
85             `WB_BUSY:         begin
86                 if(wishbone_ack_i == 1'b1) begin
87                     wishbone_stb_o <= 1'b0;
88                     wishbone_cyc_o <= 1'b0;
89                     wishbone_addr_o <= `ZeroWord;
90                     wishbone_data_o <= `ZeroWord;
91                     wishbone_we_o <= `WriteDisable;
92                     wishbone_sel_o <= 4'b0000;
93                     wishbone_state <= `WB_IDLE;
94                     if(cpu_we_i == `WriteDisable) begin
95                         rd_buf <= wishbone_data_i;
96                     end

```

```

97
98         if(stall_i != 6'b000000) begin
99             wishbone_state <= `WB_WAIT_FOR_STALL;
100         end
101     end else if(flush_i == `True_v) begin
102         wishbone_stb_o <= 1'b0;
103         wishbone_cyc_o <= 1'b0;
104         wishbone_addr_o <= `ZeroWord;
105         wishbone_data_o <= `ZeroWord;
106         wishbone_we_o <= `WriteDisable;
107         wishbone_sel_o <= 4'b0000;
108         wishbone_state <= `WB_IDLE;
109         rd_buf <= `ZeroWord;
110     end
111 end
112 `WB_WAIT_FOR_STALL:    begin
113     if(stall_i == 6'b000000) begin
114         wishbone_state <= `WB_IDLE;
115     end
116 end
117 default: begin
118 end
119 endcase
120 end    //if
121 end    //always
122
123
124 always @ (*) begin
125     if(rst == `RstEnable) begin
126         stallreq <= `NoStop;
127         cpu_data_o <= `ZeroWord;
128     end else begin
129         stallreq <= `NoStop;
130         case (wishbone_state)
131             `WB_IDLE:    begin
132                 if((cpu_ce_i == 1'b1) && (flush_i == `False_v)) begin
133                     stallreq <= `Stop;
134                     cpu_data_o <= `ZeroWord;
135                 end
136             end
137             `WB_BUSY:    begin
138                 if(wishbone_ack_i == 1'b1) begin
139                     stallreq <= `NoStop;
140                     if(wishbone_we_o == `WriteDisable) begin
141                         cpu_data_o <= wishbone_data_i;    //????

```

```

142         end else begin
143             cpu_data_o <= `ZeroWord;
144         end
145     end else begin
146         stallreq <= `Stop;
147         cpu_data_o <= `ZeroWord;
148     end
149 end
150 `WB_WAIT_FOR_STALL:    begin
151     stallreq <= `NoStop;
152     cpu_data_o <= rd_buf;
153 end
154 default: begin
155 end
156 endcase
157 end    //if
158 end    //always
159
160 endmodule

```

Listing 6: wishbone_bus_if.v

A.4 全局宏定义

```

1  //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
2  // File:      defines.v
3  // Description: OpenMIPS 处理器全局宏定义
4  //           包含以下定义组:
5  //           - 通用常量: 使能/禁用、字宽等
6  //           - 指令编码 (opcode): 算术/逻辑/移位/跳转/加载存储等
7  //           - ALU 操作码 (AluOp): 8 位, 与指令一一对应
8  //           - ALU 结果选择 (AluSel): 3 位, 选择运算单元输出
9  //           - 指令/数据存储器参数: 地址总线、数据总线宽度
10 //           - 寄存器文件参数: 32 个 32 位寄存器
11 //           - 除法器状态: DivFree/DivByZero/DivOn/DivEnd
12 //           - CP0 寄存器地址: Count/Compare/Status/Cause/EPC/PrId/
13 //           Config
14 //           - Wishbone 总线状态: WB_IDLE/WB_BUSY/WB_WAIT_FOR_FLUSHING/
15 //           WB_WAIT_FOR_STALL
16 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
17 // %
18 `define RstEnable 1'b1
19 `define RstDisable 1'b0

```

```
19 `define ZeroWord 32'h00000000
20 `define WriteEnable 1'b1
21 `define WriteDisable 1'b0
22 `define ReadEnable 1'b1
23 `define ReadDisable 1'b0
24 `define AluOpBus 7:0
25 `define AluSelBus 2:0
26 `define InstValid 1'b0
27 `define InstInvalid 1'b1
28 `define Stop 1'b1
29 `define NoStop 1'b0
30 `define InDelaySlot 1'b1
31 `define NotInDelaySlot 1'b0
32 `define Branch 1'b1
33 `define NotBranch 1'b0
34 `define InterruptAssert 1'b1
35 `define InterruptNotAssert 1'b0
36 `define TrapAssert 1'b1
37 `define TrapNotAssert 1'b0
38 `define True_v 1'b1
39 `define False_v 1'b0
40 `define ChipEnable 1'b1
41 `define ChipDisable 1'b0
42
43
44 //
45 `define EXE_AND 6'b100100
46 `define EXE_OR 6'b100101
47 `define EXE_XOR 6'b100110
48 `define EXE_NOR 6'b100111
49 `define EXE_ANDI 6'b001100
50 `define EXE_ORI 6'b001101
51 `define EXE_XORI 6'b001110
52 `define EXE_LUI 6'b001111
53
54 `define EXE_SLL 6'b000000
55 `define EXE_SLLV 6'b000100
56 `define EXE_SRL 6'b000010
57 `define EXE_SRLV 6'b000110
58 `define EXE_SRA 6'b000011
59 `define EXE_SRAV 6'b000111
60 `define EXE_SYNC 6'b001111
61 `define EXE_PREF 6'b110011
62
63 `define EXE_MOVZ 6'b001010
```

```
64 `define EXE_MOVN 6'b001011
65 `define EXE_MFHI 6'b010000
66 `define EXE_MTHI 6'b010001
67 `define EXE_MFLO 6'b010010
68 `define EXE_MTLO 6'b010011
69
70 `define EXE_SLT 6'b101010
71 `define EXE_SLTU 6'b101011
72 `define EXE_SLTI 6'b001010
73 `define EXE_SLTIU 6'b001011
74 `define EXE_ADD 6'b100000
75 `define EXE_ADDU 6'b100001
76 `define EXE_SUB 6'b100010
77 `define EXE_SUBU 6'b100011
78 `define EXE_ADDI 6'b001000
79 `define EXE_ADDIU 6'b001001
80 `define EXE_CLZ 6'b100000
81 `define EXE_CLO 6'b100001
82
83 `define EXE_MULT 6'b011000
84 `define EXE_MULTU 6'b011001
85 `define EXE_MUL 6'b000010
86 `define EXE_MADD 6'b000000
87 `define EXE_MADDU 6'b000001
88 `define EXE_MSUB 6'b000100
89 `define EXE_MSUBU 6'b000101
90
91 `define EXE_DIV 6'b011010
92 `define EXE_DIVU 6'b011011
93
94 `define EXE_J 6'b000010
95 `define EXE_JAL 6'b000011
96 `define EXE_JALR 6'b001001
97 `define EXE_JR 6'b001000
98 `define EXE_BEQ 6'b000100
99 `define EXE_BGEZ 5'b00001
100 `define EXE_BGEZAL 5'b10001
101 `define EXE_BGTZ 6'b000111
102 `define EXE_BLEZ 6'b000110
103 `define EXE_BLTZ 5'b00000
104 `define EXE_BLTZAL 5'b10000
105 `define EXE_BNE 6'b000101
106
107 `define EXE_LB 6'b100000
108 `define EXE_LBU 6'b100100
```

```
// AluOp
```

```
154 `define EXE_LUI_OP 8'b01011100
155
156 `define EXE_SLL_OP 8'b01111100
157 `define EXE_SLLV_OP 8'b00000100
158 `define EXE_SRL_OP 8'b00000010
159 `define EXE_SRLV_OP 8'b00000110
160 `define EXE_SRA_OP 8'b00000011
161 `define EXE_SRAV_OP 8'b00000111
162
163 `define EXE_MOVZ_OP 8'b00001010
164 `define EXE_MOVN_OP 8'b00001011
165 `define EXE_MFHI_OP 8'b00010000
166 `define EXE_MTHI_OP 8'b00010001
167 `define EXE_MFLO_OP 8'b00010010
168 `define EXE_MTLO_OP 8'b00010011
169
170 `define EXE_SLT_OP 8'b00101010
171 `define EXE_SLTU_OP 8'b00101011
172 `define EXE_SLTI_OP 8'b01010111
173 `define EXE_SLTIU_OP 8'b01011000
174 `define EXE_ADD_OP 8'b00100000
175 `define EXE_ADDU_OP 8'b00100001
176 `define EXE_SUB_OP 8'b00100010
177 `define EXE_SUBU_OP 8'b00100011
178 `define EXE_ADDI_OP 8'b01010101
179 `define EXE_ADDIU_OP 8'b01010110
180 `define EXE_CLZ_OP 8'b10110000
181 `define EXE_CLO_OP 8'b10110001
182
183 `define EXE_MULT_OP 8'b00011000
184 `define EXE_MULTU_OP 8'b00011001
185 `define EXE_MUL_OP 8'b10101001
186 `define EXE_MADD_OP 8'b10100110
187 `define EXE_MADDU_OP 8'b10101000
188 `define EXE_MSUB_OP 8'b10101010
189 `define EXE_MSUBU_OP 8'b10101011
190
191 `define EXE_DIV_OP 8'b00011010
192 `define EXE_DIVU_OP 8'b00011011
193
194 `define EXE_J_OP 8'b01001111
195 `define EXE_JAL_OP 8'b01010000
196 `define EXE_JALR_OP 8'b00001001
197 `define EXE_JR_OP 8'b00001000
198 `define EXE_BEQ_OP 8'b01010001
```

```
199 `define EXE_BGEZ_OP 8'b01000001
200 `define EXE_BGEZAL_OP 8'b01001011
201 `define EXE_BGTZ_OP 8'b01010100
202 `define EXE_BLEZ_OP 8'b01010011
203 `define EXE_BLTZ_OP 8'b01000000
204 `define EXE_BLTZAL_OP 8'b01001010
205 `define EXE_BNE_OP 8'b01010010
206
207 `define EXE_LB_OP 8'b11100000
208 `define EXE_LBU_OP 8'b11100100
209 `define EXE_LH_OP 8'b11100001
210 `define EXE_LHU_OP 8'b11100101
211 `define EXE_LL_OP 8'b11110000
212 `define EXE_LW_OP 8'b11100011
213 `define EXE_LWL_OP 8'b11100010
214 `define EXE_LWR_OP 8'b11100110
215 `define EXE_PREF_OP 8'b11110011
216 `define EXE_SB_OP 8'b11101000
217 `define EXE_SC_OP 8'b11111000
218 `define EXE_SH_OP 8'b11101001
219 `define EXE_SW_OP 8'b11101011
220 `define EXE_SWL_OP 8'b11101010
221 `define EXE_SWR_OP 8'b11101110
222 `define EXE_SYNC_OP 8'b00001111
223
224 `define EXE_MFC0_OP 8'b01011101
225 `define EXE_MTC0_OP 8'b01100000
226
227 `define EXE_SYSCALL_OP 8'b00001100
228
229 `define EXE_TEQ_OP 8'b00110100
230 `define EXE_TEQI_OP 8'b01001000
231 `define EXE_TGE_OP 8'b00110000
232 `define EXE_TGEI_OP 8'b01000100
233 `define EXE_TGEIU_OP 8'b01000101
234 `define EXE_TGEU_OP 8'b00110001
235 `define EXE_TLT_OP 8'b00110010
236 `define EXE_TLTI_OP 8'b01000110
237 `define EXE_TLTIU_OP 8'b01000111
238 `define EXE_TLTU_OP 8'b00110011
239 `define EXE_TNE_OP 8'b00110110
240 `define EXE_TNEI_OP 8'b01001001
241
242 `define EXE_ERET_OP 8'b01101011
243
```



```

244 `define EXE_NOP_OP      8'b00000000
245
246 //AluSel
247 `define EXE_RES_LOGIC 3'b001
248 `define EXE_RES_SHIFT 3'b010
249 `define EXE_RES_MOVE 3'b011
250 `define EXE_RES_ARITHMETIC 3'b100
251 `define EXE_RES_MUL 3'b101
252 `define EXE_RES_JUMP_BRANCH 3'b110
253 `define EXE_RES_LOAD_STORE 3'b111
254
255 `define EXE_RES_NOP 3'b000
256
257
258 // 淨 inst_rom
259 `define InstAddrBus 31:0
260 `define InstBus 31:0
261 `define InstMemNum 131071
262 `define InstMemNumLog2 17
263
264 // ¼ 淨 data_ram
265 `define DataAddrBus 31:0
266 `define DataBus 31:0
267 `define DataMemNum 131071
268 `define DataMemNumLog2 17
269 `define ByteWidth 7:0
270
271 // üĴ regfile
272 `define RegAddrBus 4:0
273 `define RegBus 31:0
274 `define RegWidth 32
275 `define DoubleRegWidth 64
276 `define DoubleRegBus 63:0
277 `define RegNum 32
278 `define RegNumLog2 5
279 `define NOPRegAddr 5'b00000
280
281 //³ • "div
282 `define DivFree 2'b00
283 `define DivByZero 2'b01
284 `define DivOn 2'b10
285 `define DivEnd 2'b11
286 `define DivResultReady 1'b1
287 `define DivResultNotReady 1'b0
288 `define DivStart 1'b1

```

```

289 `define DivStop 1'b0
290
291 //CP0%J
292 `define CP0_REG_COUNT      5'b01001      //i
293 `define CP0_REG_COMPARE    5'b01011      //i
294 `define CP0_REG_STATUS     5'b01100      //i
295 `define CP0_REG_CAUSE      5'b01101      //i
296 `define CP0_REG_EPC        5'b01110      //i
297 `define CP0_REG_PrId       5'b01111      //i
298 `define CP0_REG_CONFIG     5'b10000      //i
299
300 //wishbone      »
301 `define WB_IDLE 2'b00
302 `define WB_BUSY 2'b01
303 `define WB_WAIT_FOR_FLUSHING 2'b10
304 `define WB_WAIT_FOR_STALL 2'b11

```

Listing 7: defines.v

A.5 DDR2 控制器封装

```

1 `timescale 1ns / 1ps
2 //
3 // Design Name: Looper
4 // Module Name:    looper1_1.v
5 // Project Name: Looper Project
6 // Target Devices: Nexys4 DDR
7 // Tool versions: Vivado 2015.1
8 // Description: This project turns your Nexys4 DDR into a guitar/piano/aux
9 //               input looper. Plug input into XADC3
10 //
11 // Dependencies:
12 //
13 // 0.01 - File Created
14 // 1.0 - Finished with 8 external buttons on JC, 4 memory banks
15 // 1.1 - Changed addressing, bug fixes
16 // 1.2 - Moved to different control scheme using 4 onboard buttons, banks
17 //       doubled to 8 banks, 3 minutes each
18 //
19 // Additional Comments:
20 //
21 //

```

```

////////////////////////////////////
20 `include "defines.v"
21
22 //////////////////////////////////
23 // Module:  DDR2
24 // Description: DDR2 SDRAM 控制器 — Wishbone 从设备接口
25 //      将 Wishbone 总线读写转换为 DDR2 内存访问
26 //      内部状态机: IDLE → START → WRITE/READ → ENDING
27 //      - IDLE: 等待 Wishbone 周期 (cyc & stb)
28 //      - START: 锁存地址和读写方向
29 //      - WRITE: 写操作 (等待 ~80 周期完成)
30 //      - READ: 读操作 (等待 ~80 周期完成)
31 //      - ENDING: 事务结束, 发送 ack
32 //      依赖 Ram2Ddr 模块进行底层 DDR2 PHY 控制
33 //      复位后需等待 init_calib_complete 校准完成
34 //////////////////////////////////
35
36 module DDR2(
37     input wire wb_clk_i,
38     input wire clk_200MHz_i,
39     input wire wb_rst_i,
40     input wire wb_cyc_i,
41     input wire wb_stb_i,
42     input wire wb_we_i,
43     input wire [3:0] wb_sel_i,
44     input wire [26:0] wb_adr_i,
45     input wire [31:0] wb_dat_i,
46     output reg [31:0] wb_dat_o,
47     output reg        wb_ack_o,
48     output reg init_calib_complete,
49
50     //memory signals
51     output [12:0] ddr2_addr,
52     output [2:0] ddr2_ba,
53     output ddr2_ras_n,
54     output ddr2_cas_n,
55     output ddr2_we_n,
56     output ddr2_ck_p,
57     output ddr2_ck_n,
58     output ddr2_cke,
59     output ddr2_cs_n,
60     output [1:0] ddr2_dm,
61     output ddr2_odt,
62     inout [15:0] ddr2_dq,

```

```

63     inout    [1:0] ddr2_dqs_p,
64     inout    [1:0] ddr2_dqs_n
65 );
66
67
68
69     parameter tenhz = 10000000;
70
71     // Max_block = 64,000,000 / 8 = 8,000,000 or 0111 1010 0001 0010 0000
72     // 22:0
73     //8 banks
74
75
76     reg [22:0] max_block=0;
77     reg [26:0] timercnt=0;
78     reg [11:0] timerval=0;
79
80     wire set_max;
81     wire reset_max;
82     wire p;//Bank is playing
83     wire r;//Bank is recording
84     (* Mark_debug = "TRUE" *)reg p_r;
85     (* Mark_debug = "TRUE" *)reg r_r;
86     wire del_mem;//Clear delete flag
87     wire delete;//Delete flag
88     wire [2:0] delete_bank;//Bank to delete
89     wire [2:0] mem_bank;//Bank
90     wire write_zero;//Used when deleting
91     wire [22:0] current_block;//Block address
92     wire [3:0] buttons_db;//Debounced buttons
93     wire [7:0] active;//Bank is recorded on
94
95     wire [2:0] current_bank;
96
97     wire [26:0] mem_a_w;
98     reg [26:0] mem_a;
99     assign mem_a_w=mem_a;
100
101     //So address cycles through 0 - 1 - 2 - 3 - 4 - 5 - 6 - 7, then current
102     //block is incremented by 1 and mem_bank goes back to 0: mem_a = 8
103     reg [31:0] mem_dq_i;
104     wire [31:0] mem_dq_i_w;
105     assign mem_dq_i_w=mem_dq_i;
106     wire [31:0] mem_dq_o;

```

```

106
107     reg mem_cen;
108     reg mem_oen;
109     reg mem_wen;
110     wire mem_cen_w;
111     wire mem_oen_w;
112     wire mem_wen_w;
113     assign mem_cen_w=mem_cen;
114     assign mem_oen_w=mem_oen;
115     assign mem_wen_w=mem_wen;
116
117
118     wire mem_ub;
119     wire mem_lb;
120     wire [3:0]mem_sel;
121     assign mem_ub = 0;
122     assign mem_lb = 0;
123     assign mem_sel= ~wb_sel_i;
124
125     wire [15:0] chipTemp;
126
127     wire data_flag;
128
129     wire data_ready;
130
131     wire mix_data;
132     wire [22:0] block44KHz;
133
134 // clk_wiz_0 moved to top-level (openmips_min_sopc.v); clk_200MHz_i comes
    from port
135
136 //
    ///////////////////////////////////////////////////
137 ////      Memory instantiation
138 //
    ///////////////////////////////////////////////////
139
    //sel 0 ??? 1 ???
140 //????32????
141     wire init_calib_complete_w;
142     Ram2Ddr Ram(
143         .clk_200MHz_i      (clk_200MHz_i),
144         .rst_i             (wb_rst_i),
145         .device_temp_i     (chipTemp[11:0]),

```

```

146     .init_calib_complete_o (init_calib_complete_w),
147     // RAM interface
148     .ram_a                (mem_a_w),
149     .ram_dq_i             (mem_dq_i_w),
150     .ram_dq_o             (mem_dq_o),
151     .ram_cen              (mem_cen_w),
152     .ram_oen              (mem_oen_w),
153     .ram_wen              (mem_wen_w),
154     .ram_ub               (mem_ub),
155     .ram_lb               (mem_lb),
156     .ram_sel              (mem_sel),
157     // DDR2 interface
158     .ddr2_addr            (ddr2_addr),
159     .ddr2_ba              (ddr2_ba),
160     .ddr2_ras_n           (ddr2_ras_n),
161     .ddr2_cas_n           (ddr2_cas_n),
162     .ddr2_we_n           (ddr2_we_n),
163     .ddr2_ck_p           (ddr2_ck_p),
164     .ddr2_ck_n           (ddr2_ck_n),
165     .ddr2_cke             (ddr2_cke),
166     .ddr2_cs_n            (ddr2_cs_n),
167     .ddr2_dm              (ddr2_dm),
168     .ddr2_odt            (ddr2_odt),
169     .ddr2_dq              (ddr2_dq),
170     .ddr2_dqs_p           (ddr2_dqs_p),
171     .ddr2_dqs_n           (ddr2_dqs_n)
172 );

```

```

173
174
175
176 //

```

```

////////////////////////////////////

```

```

177 ////    Data in latch

```

```

178 //

```

```

////////////////////////////////////

```

```

179

```

```

180

```

```

181

```

```

182     //Data in is assigned the latched data input from sound_data, or .5V
183     (16'h7444) if write zero is on

```

```

183

```

```

184

```

```

185

```

```

186 //-----wishbone
187
188 parameter IDLE = 5'd0;
189 parameter START = 5'd1;
190 parameter WRITE = 5'd2;
191 parameter READ = 5'd3;
192 parameter WAIT = 5'd4;
193 parameter ENDING = 5'd5;
194
195 reg [4:0] state;
196 reg [4:0] next_state;
197 reg [15:0] wait_count;
198 // change state
199 always @(posedge wb_clk_i or posedge wb_rst_i) begin
200     if(wb_rst_i)begin
201         state<=IDLE;
202         wb_ack_o<=1'b0;
203         init_calib_complete<=1'b0;
204     end
205     else begin
206         init_calib_complete<=init_calib_complete_w;
207         if(wb_cyc_i&wb_stb_i)begin
208             state<=next_state;
209             if(state==ENDING&&!wb_ack_o)begin
210                 wb_ack_o<=1'b1;
211             end
212         end
213         else begin
214             state<=IDLE;
215             wb_ack_o<=1'b0;
216         end
217     end
218
219 //set sign by state
220 always@(posedge wb_clk_i or posedge wb_rst_i)begin
221     if(wb_rst_i)begin
222         next_state<=IDLE;
223         mem_cen<=1'b1;
224         mem_oen<=1'b1;
225         mem_wen<=1'b1;
226     end
227     else begin
228         case(state)
229             IDLE:

```

```
230         begin
231             mem_cen<=1'b1;
232             mem_oen<=1'b1;
233             mem_wen<=1'b1;
234             wait_count<=16'b0;
235             next_state<=START;
236         end
237     START:
238     begin
239         mem_a<=wb_adr_i;
240         if(wb_we_i)
241         begin
242             next_state<=WRITE;
243         end
244         else
245         begin
246             next_state<=READ;
247         end
248     end
249     WRITE:
250     begin
251         mem_cen<=1'b0;
252         mem_oen<=1'b1;
253         mem_wen<=1'b0;
254         mem_a<=wb_adr_i;
255         mem_dq_i<=wb_dat_i;
256         wait_count<=wait_count+16'd1;
257         if(wait_count>=16'd80)
258         begin
259             mem_cen<=1'b1;
260             mem_oen<=1'b1;
261             mem_wen<=1'b1;
262             next_state<=ENDING;
263         end
264     end
265     READ:
266     begin
267         mem_cen<=1'b0;
268         mem_oen<=1'b0;
269         mem_wen<=1'b1;
270         mem_a<=wb_adr_i;
271         wait_count<=wait_count+16'd1;
272         if(wait_count>=16'd80)
273         begin
274             mem_cen<=1'b1;
```



```

275         mem_oen<=1'b1;
276         mem_wen<=1'b1;
277         wb_dat_o<=mem_dq_o;
278         next_state<=ENDING;
279     end
280 end
281 ENDING:
282 begin
283 end
284
285 endcase
286 end
287
288 end
289
290
291
292
293 //
-----
294
295 endmodule

```

Listing 8: DDR2.v

A.6 引脚约束

```

1 ## Clock signal
2 set_property -dict {PACKAGE_PIN E3 IOSTANDARD LVCMOS33} [get_ports clk_in]
3
4 ## Switches
5 set_property -dict {PACKAGE_PIN J15 IOSTANDARD LVCMOS33} [get_ports
    flash_continue]
6 set_property -dict {PACKAGE_PIN L16 IOSTANDARD LVCMOS33} [get_ports {gpio_i
    [1]}]
7 set_property -dict {PACKAGE_PIN M13 IOSTANDARD LVCMOS33} [get_ports {gpio_i
    [2]}]
8 set_property -dict {PACKAGE_PIN R15 IOSTANDARD LVCMOS33} [get_ports {gpio_i
    [3]}]
9 set_property -dict {PACKAGE_PIN R17 IOSTANDARD LVCMOS33} [get_ports {gpio_i
    [4]}]
10 set_property -dict {PACKAGE_PIN T18 IOSTANDARD LVCMOS33} [get_ports {gpio_i
    [5]}]

```

```

11 set_property -dict {PACKAGE_PIN U18 IOSTANDARD LVCMOS33} [get_ports {gpio_i
    [6]]}
12 set_property -dict {PACKAGE_PIN R13 IOSTANDARD LVCMOS33} [get_ports {gpio_i
    [7]]}
13 set_property -dict {PACKAGE_PIN T8 IOSTANDARD LVCMOS18} [get_ports {gpio_i
    [8]]}
14 set_property -dict {PACKAGE_PIN U8 IOSTANDARD LVCMOS18} [get_ports {gpio_i
    [9]]}
15 set_property -dict {PACKAGE_PIN R16 IOSTANDARD LVCMOS33} [get_ports {gpio_i
    [10]]}
16 set_property -dict {PACKAGE_PIN T13 IOSTANDARD LVCMOS33} [get_ports {gpio_i
    [11]]}
17 set_property -dict {PACKAGE_PIN H6 IOSTANDARD LVCMOS33} [get_ports {gpio_i
    [12]]}
18 set_property -dict {PACKAGE_PIN U12 IOSTANDARD LVCMOS33} [get_ports {gpio_i
    [13]]}
19 set_property -dict {PACKAGE_PIN U11 IOSTANDARD LVCMOS33} [get_ports {gpio_i
    [14]]}
20 set_property -dict {PACKAGE_PIN V10 IOSTANDARD LVCMOS33} [get_ports {gpio_i
    [15]]}
21
22
23 ## LEDs
24 set_property -dict {PACKAGE_PIN H17 IOSTANDARD LVCMOS33} [get_ports {
    debug_led[0]}]
25 set_property -dict {PACKAGE_PIN K15 IOSTANDARD LVCMOS33} [get_ports {
    debug_led[1]}]
26 set_property -dict {PACKAGE_PIN J13 IOSTANDARD LVCMOS33} [get_ports {
    debug_led[2]}]
27 set_property -dict {PACKAGE_PIN N14 IOSTANDARD LVCMOS33} [get_ports {
    debug_led[3]}]
28 set_property -dict {PACKAGE_PIN R18 IOSTANDARD LVCMOS33} [get_ports {
    debug_led[4]}]
29 set_property -dict {PACKAGE_PIN V17 IOSTANDARD LVCMOS33} [get_ports {
    debug_led[5]}]
30 set_property -dict {PACKAGE_PIN U17 IOSTANDARD LVCMOS33} [get_ports {
    debug_led[6]}]
31 set_property -dict {PACKAGE_PIN U16 IOSTANDARD LVCMOS33} [get_ports {
    debug_led[7]}]
32 set_property -dict {PACKAGE_PIN V16 IOSTANDARD LVCMOS33} [get_ports {
    debug_led[8]}]
33 #set_property -dict { PACKAGE_PIN N14 IOSTANDARD LVCMOS33 } [get_ports {
    gpio_o[19] }]; #IO_L15N_T2_DQS_DOUT_CS0_B_14 Sch=led[11]
34 #set_property -dict { PACKAGE_PIN N14 IOSTANDARD LVCMOS33 } [get_ports {
    gpio_o[19] }]; #IO_L15N_T2_DQS_DOUT_CS0_B_14 Sch=led[11]

```

```

35 #set_property -dict { PACKAGE_PIN R18      IOSTANDARD LVCMOS33 } [get_ports {
    gpio_o[20] }]; #IO_L16N_T2_A15_D31_14 Sch=led[8]
36 #set_property -dict { PACKAGE_PIN V17      IOSTANDARD LVCMOS33 } [get_ports {
    gpio_o[21] }]; #IO_L14N_T2_SRCC_14 Sch=led[9]
37 #set_property -dict { PACKAGE_PIN U17      IOSTANDARD LVCMOS33 } [get_ports {
    gpio_o[22] }]; #IO_L22P_T3_A05_D21_14 Sch=led[10]
38 #set_property -dict { PACKAGE_PIN U16      IOSTANDARD LVCMOS33 } [get_ports {
    gpio_o[23] }]; #IO_L15N_T2_DQS_DOUT_CS0_B_14 Sch=led[11]
39 #set_property -dict { PACKAGE_PIN V16      IOSTANDARD LVCMOS33 } [get_ports {
    gpio_o[24] }]; #IO_L16N_T2_A15_D31_14 Sch=led[8]
40 #set_property -dict { PACKAGE_PIN T15      IOSTANDARD LVCMOS33 } [get_ports {
    gpio_o[25] }]; #IO_L14N_T2_SRCC_14 Sch=led[9]
41 #set_property -dict { PACKAGE_PIN U14      IOSTANDARD LVCMOS33 } [get_ports {
    gpio_o[26] }]; #IO_L22P_T3_A05_D21_14 Sch=led[10]
42 #set_property -dict { PACKAGE_PIN T16      IOSTANDARD LVCMOS33 } [get_ports {
    gpio_o[27] }]; #IO_L15N_T2_DQS_DOUT_CS0_B_14 Sch=led[11]
43 #set_property -dict { PACKAGE_PIN V15      IOSTANDARD LVCMOS33 } [get_ports {
    gpio_o[28] }]; #IO_L16P_T2_CSI_B_14 Sch=led[12]
44 #set_property -dict { PACKAGE_PIN V14      IOSTANDARD LVCMOS33 } [get_ports {
    gpio_o[29] }]; #IO_L22N_T3_A04_D20_14 Sch=led[13]
45 #set_property -dict { PACKAGE_PIN V12      IOSTANDARD LVCMOS33 } [get_ports {
    gpio_o[30] }]; #IO_L20N_T3_A07_D23_14 Sch=led[14]
46 #set_property -dict { PACKAGE_PIN V11      IOSTANDARD LVCMOS33 } [get_ports {
    gpio_o[31] }]; #IO_L21N_T3_DQS_A06_D22_14 Sch=led[15]
47
48
49 ## SEG
50 #set_property -dict {PACKAGE_PIN H15 IOSTANDARD LVCMOS33} [get_ports {
    gpio_o[15]}}]
51 #set_property -dict {PACKAGE_PIN L18 IOSTANDARD LVCMOS33} [get_ports {
    gpio_o[14]}}]
52 #set_property -dict {PACKAGE_PIN T11 IOSTANDARD LVCMOS33} [get_ports {
    gpio_o[13]}}]
53 #set_property -dict {PACKAGE_PIN P15 IOSTANDARD LVCMOS33} [get_ports {
    gpio_o[12]}}]
54 #set_property -dict {PACKAGE_PIN K13 IOSTANDARD LVCMOS33} [get_ports {
    gpio_o[11]}}]
55 #set_property -dict {PACKAGE_PIN K16 IOSTANDARD LVCMOS33} [get_ports {
    gpio_o[10]}}]
56 #set_property -dict {PACKAGE_PIN R10 IOSTANDARD LVCMOS33} [get_ports {
    gpio_o[9]}}]
57 #set_property -dict {PACKAGE_PIN T10 IOSTANDARD LVCMOS33} [get_ports {
    gpio_o[8]}}]
58 #set_property -dict {PACKAGE_PIN U13 IOSTANDARD LVCMOS33} [get_ports {
    gpio_o[7]}}]

```

```
59 #set_property -dict {PACKAGE_PIN K2 IOSTANDARD LVCMOS33} [get_ports {  
    gpio_o[6]]  
60 #set_property -dict {PACKAGE_PIN T14 IOSTANDARD LVCMOS33} [get_ports {  
    gpio_o[5]]  
61 #set_property -dict {PACKAGE_PIN P14 IOSTANDARD LVCMOS33} [get_ports {  
    gpio_o[4]]  
62 #set_property -dict {PACKAGE_PIN J14 IOSTANDARD LVCMOS33} [get_ports {  
    gpio_o[3]]  
63 #set_property -dict {PACKAGE_PIN T9 IOSTANDARD LVCMOS33} [get_ports {  
    gpio_o[2]]  
64 #set_property -dict {PACKAGE_PIN J18 IOSTANDARD LVCMOS33} [get_ports {  
    gpio_o[1]]  
65 #set_property -dict {PACKAGE_PIN J17 IOSTANDARD LVCMOS33} [get_ports {  
    gpio_o[0]]  
66  
67 #set_property -dict {PACKAGE_PIN U13 IOSTANDARD LVCMOS33} [get_ports {  
    gpio_o[15]]  
68 #set_property -dict {PACKAGE_PIN K2 IOSTANDARD LVCMOS33} [get_ports {gpio_o  
    [14]]  
69 #set_property -dict {PACKAGE_PIN T14 IOSTANDARD LVCMOS33} [get_ports {  
    gpio_o[13]]  
70 #set_property -dict {PACKAGE_PIN P14 IOSTANDARD LVCMOS33} [get_ports {  
    gpio_o[12]]  
71 #set_property -dict {PACKAGE_PIN J14 IOSTANDARD LVCMOS33} [get_ports {  
    gpio_o[11]]  
72 #set_property -dict {PACKAGE_PIN T9 IOSTANDARD LVCMOS33} [get_ports {gpio_o  
    [10]]  
73 #set_property -dict {PACKAGE_PIN J18 IOSTANDARD LVCMOS33} [get_ports {  
    gpio_o[9]]  
74 #set_property -dict {PACKAGE_PIN J17 IOSTANDARD LVCMOS33} [get_ports {  
    gpio_o[8]]  
75 set_property -dict {PACKAGE_PIN H15 IOSTANDARD LVCMOS33} [get_ports {gpio_o  
    [7]]  
76 set_property -dict {PACKAGE_PIN L18 IOSTANDARD LVCMOS33} [get_ports {gpio_o  
    [6]]  
77 set_property -dict {PACKAGE_PIN T11 IOSTANDARD LVCMOS33} [get_ports {gpio_o  
    [5]]  
78 set_property -dict {PACKAGE_PIN P15 IOSTANDARD LVCMOS33} [get_ports {gpio_o  
    [4]]  
79 set_property -dict {PACKAGE_PIN K13 IOSTANDARD LVCMOS33} [get_ports {gpio_o  
    [3]]  
80 set_property -dict {PACKAGE_PIN K16 IOSTANDARD LVCMOS33} [get_ports {gpio_o  
    [2]]  
81 set_property -dict {PACKAGE_PIN R10 IOSTANDARD LVCMOS33} [get_ports {gpio_o  
    [1]]
```

```
82 set_property -dict {PACKAGE_PIN T10 IOSTANDARD LVCMOS33} [get_ports {gpio_o
    [0]}]
83
84
85
86 ## Buttons
87 set_property -dict {PACKAGE_PIN C12 IOSTANDARD LVCMOS33} [get_ports rst_n]
88 set_property -dict {PACKAGE_PIN N17 IOSTANDARD LVCMOS33} [get_ports {gpio_i
    [0]}]
89
90
91 ## USB-RS232 Interface
92 set_property -dict {PACKAGE_PIN C4 IOSTANDARD LVCMOS33} [get_ports uart_in]
93 set_property -dict {PACKAGE_PIN D4 IOSTANDARD LVCMOS33} [get_ports uart_out
    ]
94
95
96 ## Quad SPI Flash
97 set_property -dict {PACKAGE_PIN K17 IOSTANDARD LVCMOS33} [get_ports sdo]
98 set_property -dict {PACKAGE_PIN K18 IOSTANDARD LVCMOS33} [get_ports sdi]
99 set_property -dict {PACKAGE_PIN L14 IOSTANDARD LVCMOS33} [get_ports wp_n]
100 set_property -dict {PACKAGE_PIN M14 IOSTANDARD LVCMOS33} [get_ports hld_n]
101 set_property -dict {PACKAGE_PIN L13 IOSTANDARD LVCMOS33} [get_ports cs_n]
102
103 ## CDC: Wishbone clock (clk_wiz_0) and MIG PLL clock are asynchronous
104 set_clock_groups -asynchronous -group [get_clocks clk_out1_clk_wiz_0] -
    group [get_clocks clk_pll_i]
```

Listing 9: nexys4ddr.xdc